

UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA

Dipartimento di Informatica, Sistemistica e Comunicazione

Corso di Laurea Triennale in Informatica

Appunti del corso

Introduzione all'Intelligenza Artificiale

Luca Giandomenico

Anno Accademico 2024/2025

Indice

1	Introduzione al corso	5
1.1	Concetto di agente intelligente, architetture e agente ambiente	5
1.1.1	Agente intelligente	5
1.1.2	Razionalità di un agente	5
1.1.3	La struttura degli agenti	6
1.1.4	Gli agenti e l'ambiente	6
1.2	Classificazione degli agenti	6
1.2.1	Agenti basati su riflessi semplici	6
1.2.2	Agenti basati su un modello con riflessi	7
1.2.3	Agenti basati su obiettivo e su un modello	7
1.3	Sistema di ricerca	8
1.4	Problema di ricerca	8
1.5	Agenti basati su utilità	8
1.6	Caratteristiche degli ambienti di lavoro	8
2	Risolvere i problemi con la ricerca	9
2.1	Problemi di ricerca e soluzioni	9
2.2	Algoritmi di ricerca	10
2.3	Strutture dati per la ricerca	10
2.4	Strategie di ricerca non informate	11
2.5	Breadth-first search	11
2.6	Uniform cost search	11
2.7	Depth-first search	11
2.8	Iterative deepening search	11
2.9	Backtracking search	11
2.10	Riepilogo algoritmi di ricerca	12
2.11	Strategie di ricerca informate (euristiche)	12
2.12	Greedy search	12
2.13	A* search	12
3	Classical planning	13
3.1	Definizione	13
3.2	STRIPS (Stanford Research Institute Problem Solver)	13
3.3	Struttura di un problema STRIP	13
3.4	PDDL (Planning Domain Description Language)	14
4	Probabilistic planning	14
4.1	Markov localization	14
4.2	Filtro ricorsivo di Bayes	15
4.2.1	Notazioni	15
4.2.2	Fase di predizione	15
4.2.3	Fase di aggiornamento	15
4.2.4	Passo ricorsivo	15
4.3	Processo decisionale di Markov	15
4.4	Politiche	16

4.5	Fattore di sconto	16
4.6	Equazione di Bellman	17
4.7	Funzione di utilità Q	17
4.8	Politica ottimale	17
4.9	Value iteration	18
4.9.1	ContraZIONE	18
4.10	Policy iteration	18
4.11	SARSA λ -learning (State, Action, Reward, State, Action)	18
5	Intelligenza artificiale simbolica	19
5.1	KRR: Knowledge Representation Reasoning	19
5.2	Base di conoscenza	19
5.3	Differenze tra KB e database	19
5.4	Ragionamento e inferenze	19
5.5	KRR e logica del primo ordine	19
5.6	Principali approcci all'intelligenza artificiale basati su KRR	20
5.7	Knowledge graphs	20
5.8	Approcci alla rappresentazione di knowledge graphs	20
6	RDF (Resource Description Framework)	21
6.1	Semantic web stack	21
6.2	Principi base di RDF	22
6.2.1	URL e URN	22
6.2.2	HTTP content negotiation	22
6.2.3	URIfied graph and triples	22
6.3	RDF Turtle/N-Triples	23
6.4	RDF linked data	23
6.5	Vocabolari RDF	23
7	SPARQL	23
7.1	SPARQL basics	24
7.2	Pattern Matching in SPARQL	24
7.3	Basic graph patterns	25
7.3.1	Complex graph patterns	25
7.4	Path expression	25
7.4.1	Path evaluation	25
7.5	Query components	26
8	RDFS (RDF Schema)	26
8.1	RDB Vs RDF	26
8.2	Primitive in RDFS	26
8.3	Interrogazioni e inferenze con SPARQL	27
8.4	Limitazioni di RDFS	27

9	OWL (Ontology Web Language)	27
9.1	Ontologie in OWL	27
9.2	Struttura e profili di OWL 2	28
9.3	Class axioms	28
9.4	Axioms about individuals	28
9.5	OWL properties	28
9.6	Property axioms	29
9.7	Struttura di una classe OWL	29
9.8	Completezza delle knowledge bases	30
9.9	Description logics	30
9.10	Costruttori	30
9.11	Sintassi e semantica	31
9.11.1	Interpretazione	31
10	Intelligenza artificiale sub-simbolica	32
10.1	Introduzione all'apprendimento automatico	32
10.2	Machine learning Vs traditional programming	33
10.3	Supervised machine learning	34
10.4	Unsupervised machine learning	34
10.5	Batch Vs online learning	35
10.6	Instance-based learning	35
11	Supervised Learning	36
11.1	Concetti Base	36
11.2	Supervised learning process	36
11.2.1	Assunzione fondamentale nell'apprendimento	36
11.3	Decision tree induction	37
11.4	Algoritmo per l'apprendimento dell'albero decisionale	37
11.4.1	Come scegliere un attributo per partizionare il dataset	38
11.5	Teoria dell'informazione	39
11.5.1	Entropia	39
11.5.2	Information gain	39
11.5.3	Gestione degli attributi continui	39
11.6	Overfitting negli alberi decisionali	39
11.7	Valutazione dei classificatori	40
12	Metriche di valutazione	40
12.1	N-fold cross-validation	41
12.1.1	Leave-one-out cross-validation	41
12.2	Validation set	41
12.3	Misure di classificazione	41
12.4	Metriche Precision e Recall	41
12.4.1	Precision	42
12.4.2	Recall	42
12.5	F_1 -Score	42
12.6	Receiver operating characteristic curve (ROC curve)	42
12.6.1	Area sotto la curva (AUC)	43

12.7 k-Nearest Neighbor Classification (kNN)	43
12.7.1 Caratteristiche di kNN	44
12.8 Ensemble methods	44
13 Machine learning	45
13.1 Percettrone	45
13.2 Classificazione lineare	45
13.3 Aggiornamento dei pesi	46
13.4 Regole di decisione multi-classe	46
13.5 Percettrone multi classe	47
13.6 Proprietà del percettroni	47
13.7 Problemi del percettrone	48
13.7.1 Dati non separabili: decisione deterministica	48
13.7.2 Dati non separabili: decisione probabilistica	48
13.8 Decisioni deterministiche dei percettroni	48
13.8.1 Esempio sulle decisioni probabilistiche binarie	49
13.9 Decisioni probabilistiche dei percettroni	49
13.9.1 Esempio sulle decisioni probabilistiche multi-classe	50
14 Deep learning	50
14.1 Regressione logistica	50
14.1.1 Funzione di perdita	51
14.2 Regressione logistica multi-classe	51
14.3 Rete neurale ad un livello	51
14.4 Funzioni di attivazione comuni	53
15 Addestramento delle reti neurali	53
15.1 Loss ottimizzazione	53
15.2 backpropagation	54
15.3 Varianti di gradient descent	54
15.4 Problematiche di SGD	55
15.5 SGD + momentum	55
15.6 AdaGrad	56
15.7 Learning rate	57
15.8 Regolarizzazione	58
15.9 Overfitting	60
16 Scelta degli iperparametri	60
16.1 Grid search Vs random search	61
17 Reti neurali convoluzionali CNN	62
17.1 Struttura spaziale	62
17.2 Computazione	62
17.3 Convolutional layer	63
17.4 Transfer learning	64

18 Apprendimento non supervisionato	65
18.1 Clustering	65
18.2 Clustering Vs classificazione	66
18.3 K-means	67
18.4 Punti di forza e di debolezza di k-means	69
18.5 Gestione degli outlier	69
18.6 Valutazione dei cluster	69
18.7 Clustering basato su densità	70
18.8 Definizione di densità	70
18.9 Potenzialità e difetti di DBSCAN	71

1 Introduzione al corso

1.1 Concetto di agente intelligente, architetture e agente ambiente

1.1.1 Agente intelligente

Un agente è qualunque cosa che può essere vista come percepente il suo ambiente attraverso dei sensori. Gli agenti possono essere di vario tipo, umani, robotici oppure software. Una sequenza percettiva di un agente è la cronologia completa di tutto ciò che l'agente ha mai percepito. La scelta di un'azione da parte di un agente in un dato istante può dipendere dall'intera sequenza percettiva osservata fino a quel momento, ma non da qualcosa che non ha percepito.

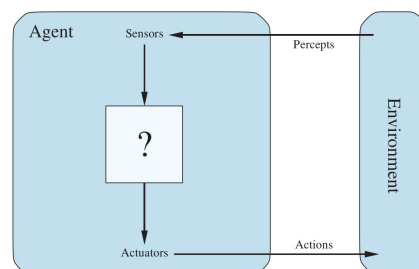


Figura 1: Rappresentazione di un agente semplice

1.1.2 Razionalità di un agente

Un agente razionale è definito come "agente che fa la cosa giusta", la razionalità di un agente tuttavia non consiste solo nel fare la cosa giusta occasionalmente ma di massimizzare costantemente la performance attesa in base alle informazioni disponibili.

La razionalità di un agente viene definita in base a quattro fattori:

- La misura di performance che definisce il successo
- La conoscenza pregressa dell'ambiente da parte dell'agente
- Le azioni che l'agente può eseguire
- La sequenza di percezioni dell'agente fino a quel momento

1.1.3 La struttura degli agenti

Il compito dell'intelligenza artificiale è progettare un programma agente che implementi la funzione agente, ovvero la mappatura dalle percezioni alle azioni. Si presume che questo programma funzioni su un dispositivo informatico con sensori e attuatori fisici, che chiamiamo architettura dell'agente:

$$\text{agente} = \text{architettura} + \text{programma}$$

Il programma scelto deve essere appropriato per l'architettura. L'architettura fornisce le percezioni dai sensori al programma, esegue il programma e trasmette le scelte di azione del programma agli attuatori man mano che vengono generate

1.1.4 Gli agenti e l'ambiente

La forma più generale di agente è una funzione che mappa l'insieme potenza dell'insieme $\mathcal{P}$, $\mathcal{P}$ rappresenta l'insieme di tutte le possibili percezioni istantanee.

$$[f : \mathcal{P}^* \rightarrow \mathcal{A}] \quad (1)$$

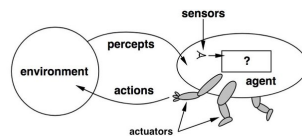


Figura 2: Rappresentazione dell'interazione tra agente e ambiente

1.2 Classificazione degli agenti

Gli agenti possono essere classificati in diverse classi in base alla loro architettura interna:

- Agenti basati su riflessi semplici
- Agenti basati su un modello con riflessi
- Agenti basati su un obiettivo e su un modello
- Agenti basati su utilità

1.2.1 Agenti basati su riflessi semplici

Un agente basato su riflessi semplici è un agente nel quale c'è comunicazione con il mondo esterno attraverso i sensori che rappresentano il modo di avere input e attuatori che cambiano lo stato del mondo. Dentro l'agente i sensori vanno a realizzare una vista che rappresenta il mondo in quel preciso istante. L'agente deve in seguito scegliere l'azione da eseguire per cui ha a disposizione solo delle regole condizione e azione. Gli agenti basati su riflessi semplici non sono muniti di stato.

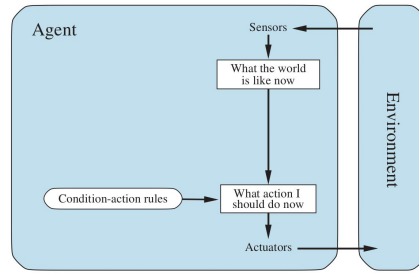


Figura 3: Agente basato su riflessi semplici

1.2.2 Agenti basati su un modello con riflessi

Gli agenti basati su un modello con riflessi sono agenti intelligenti che prendono decisioni basate su un modello interno del loro ambiente. Questo modello rappresenta una sorta di comprensione di come l'ambiente funziona, permettendo all'agente di fare previsioni sul futuro stato del mondo in risposta alle sue azioni. A differenza di un agente basato solo su riflessi semplici (che reagisce direttamente agli stimoli attuali senza tenere conto della storia precedente), un agente basato su un modello tiene traccia dello stato corrente e usa questa informazione per prendere decisioni.

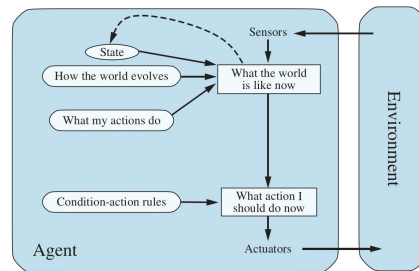


Figura 4: Agente basato su un modello con riflessi

1.2.3 Agenti basati su obiettivo e su un modello

Un goal-based agent è un agente che agisce con l'obiettivo di raggiungere uno o più obiettivi definiti. L'agente può non avere necessariamente un modello dettagliato del mondo, ma sceglie le azioni in base al fatto che lo avvicinino o meno a un obiettivo specifico. Il suo funzionamento può essere paragonabile ad un albero di ricerca in cui si prende lo stato attuale come radice dell'albero e tutte le possibili azioni sono rappresentate dalle foglie dell'albero. Per ogni stato raggiungibile è sempre possibile approfondire l'albero in base a come il mondo si evolve. L'agente naviga l'albero in modo da trovare una configurazione per la quale l'obiettivo sia verificato.

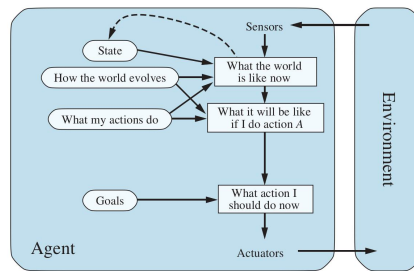


Figura 5: Agente basato su un obiettivo e su un modello

1.3 Sistema di ricerca

Un sistema di ricerca è un metodo utilizzato per esplorare uno spazio di possibili soluzioni per risolvere un problema, in particolare quando il problema può essere rappresentato come la ricerca di un cammino o di una soluzione all'interno di un ambiente complesso.

1.4 Problema di ricerca

Un problema di ricerca è una situazione in cui un agente o un algoritmo deve trovare una soluzione all'interno di uno spazio di possibili soluzioni, partendo da uno stato iniziale e cercando di raggiungere uno stato obiettivo attraverso una serie di azioni o passi intermedi.

1.5 Agenti basati su utilità

Gli utility-based agents sono un tipo avanzato di agenti che non solo perseguono obiettivi, ma cercano di ottimizzare i risultati, massimizzando una funzione di utilità. La funzione di utilità assegna un punteggio numerico a ogni stato possibile dell'ambiente, quantificando quanto sia desiderabile o vantaggioso uno stato rispetto agli altri.

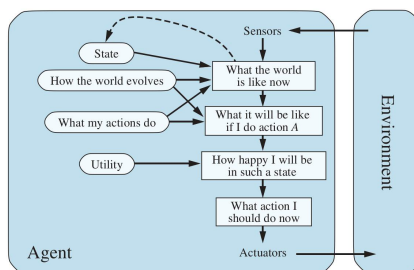


Figura 6: Agente basato su utilità

1.6 Caratteristiche degli ambienti di lavoro

- **Ambienti accessibili Vs. ambienti inaccessibili:**
un ambiente è accessibile quando l'agente può ottenere informazioni complete, accurate e aggiornate sullo stato dell'ambiente
- **Ambienti deterministici Vs. ambienti non deterministici:**
in un ambiente deterministico, ogni azione ha un singolo effetto garantito, senza incertezze sul risultato.

- **Ambienti episodici Vs. ambienti sequenziali:**

in un ambiente episodico, l'esperienza dell'agente si divide in passi atomici in cui percepisce e compie un'unica azione. Il vantaggio per lo sviluppatore è che l'agente prende decisioni solo in base all'episodio corrente, senza preoccuparsi delle interazioni future.

- **Ambienti statici Vs. ambienti dinamici:**

un ambiente statico non cambia se non per le azioni dell'agente, mentre un ambiente dinamico può essere modificato da processi esterni, indipendenti dall'agente.

- **Ambienti discreti Vs. ambienti continui:**

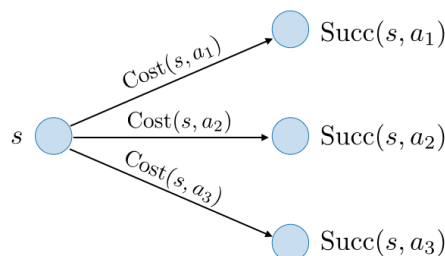
un ambiente è discreto se ha un numero finito di azioni e percezioni come per esempio il gioco degli scacchi, a differenza della guida di un taxi che invece rappresenta un ambiente continuo.

2 Risolvere i problemi con la ricerca

2.1 Problemi di ricerca e soluzioni

Un problema di ricerca può essere definito formalmente come segue:

- **Spazio degli stati:** un insieme di possibili stati in cui si trova l'ambiente.
- **Stato iniziale:** lo stato in cui l'agente si trova inizialmente.
- **Uno o un insieme di stati obiettivo:** a volte esiste un singolo stato obiettivo, altre volte lo stato obiettivo è formato da un insieme di stati mentre altre volte è definito da una proprietà che si applica a molti stati.
- **Le possibili azioni per un agente:** dato uno stato s $ACTION(s)$ ritorna un insieme finito di azioni che possono essere eseguite in s .
- **Un modello di transizione:** descrive che operazione viene effettuata da un'azione.
- **Una funzione di costo dell'azione:** è denotata da $ACTION(s, a, s')$ e viene utilizzata da un agente per misurare la propria prestazione.



Un insieme di azioni formano un **path** (cammino). Una **soluzione** è un cammino dallo stato iniziale allo stato obiettivo. Supponendo i costi delle azioni additivi, il costo totale di un cammino è la somma dei costi delle singole azioni. Una **Soluzione ottimale** ha il cammino di costo minore tra tutte le possibili soluzioni.

2.2 Algoritmi di ricerca

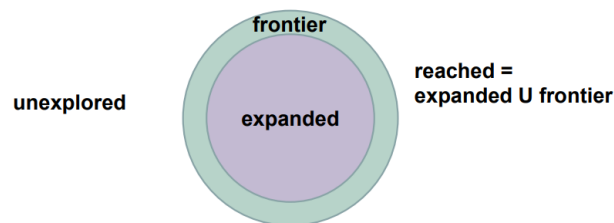
Un algoritmo di ricerca prende in input un problema di ricerca e restituisce una soluzione o una indicazione di fallimento nella ricerca. Si considerano degli algoritmi che operano attraverso un albero di ricerca nello spazio degli stati, formando vari percorsi a partire dallo stato iniziale cercando di trovare un percorso che raggiunga uno stato obiettivo. Ogni nodo dell'albero di ricerca corrisponde a uno stato nello spazio degli stati e gli archi nell'albero di ricerca corrispondono a delle azioni, la radice dell'albero corrisponde allo stato iniziale.

2.3 Strutture dati per la ricerca

Gli algoritmi di ricerca richiedono una struttura dati per tenere traccia dell'albero di ricerca. Un nodo nell'albero è rappresentato da una struttura dati con quattro componenti:

- `node.STATE`: lo stato in cui si trova il nodo.
- `node.PARENT`: il nodo nell'albero che ha generato il nodo corrente.
- `node.ACTION`: l'azione che è stata applicata allo stato del genitore per generare questo nodo.
- `node.PATH-COST`: il costo totale del percorso dallo stato iniziale a questo nodo.

La **frontiera** separa la regione espansa dalla regione inesplorata del grafo dello spazio degli stati.



È necessaria una struttura dati per memorizzare la **frontiera**, la scelta appropriata è una coda.

- `IS-EMPTY(frontier)`: restituisce vero solo se non ci sono nodi nella frontiera.
- `POP(frontier)`: rimuove il nodo superiore dalla frontiera e lo restituisce.
- `TOP(frontier)`: restituisce (ma non rimuove) il nodo superiore della frontiera.
- `ADD(node, frontier)`: inserisce il nodo nel suo posto appropriato nella coda.

Negli algoritmi di ricerca vengono utilizzate tre tipi di code:

1. **Priority queue**: esegue una POP del primo nodo con il costo minimo facendo riferimento ad una funzione di valutazione f . Viene usata nella best-first search.

2. **FIFO queue**: esegue una POP del primo nodo che è stato inserito nella coda. Viene utilizzata nella breadth-first search.
3. **LIFO queue**: conosciuta anche come **stack**, esegue una POP dell'ultimo nodo aggiunto alla coda. Viene utilizzata nella depth-first search.

2.4 Strategie di ricerca non informate

Una strategia di ricerca non informata non utilizza alcuna informazione aggiuntiva oltre alla struttura del problema stesso per esplorare uno spazio di stati.

2.5 Breadth-first search

L'algoritmo breadth-first search parte dal nodo radice ed esplora la sua frontiera, se nella frontiera trova la soluzione allora si arresta, altrimenti ripete lo stesso procedimento per ogni nodo della frontiera. La soluzione presenta d azioni e quindi vengono esplorati $O(b^d)$ nodi.

2.6 Uniform cost search

L'algoritmo uniform-cost search utilizza la funzione $W(s, a)$ per esplorare i cammini di costo minimo. Partendo da s_1 esplora la sua frontiera, successivamente esplora il nodo il cui cammino da s_1 ha costo minore. Ripete questa procedura fino ad esplorare tutto b entro una profondità D esplorando sempre i nodi con costo minore indipendentemente dalla frontiera a cui appartengono.

2.7 Depth-first search

L'algoritmo depth-first search parte da un nodo radice e visita il nodo. Da ogni nodo visitato, DFS esplora i nodi adiacenti non ancora visitati, continuando a scendere lungo i rami fino a quando non ci sono più nodi da visitare. Quando un ramo è completamente esplorato, l'algoritmo torna indietro al nodo precedente per esplorare altri rami.

2.8 Iterative deepening search

L'algoritmo iterative deepening search combina DFS e BFS per ottenere i vantaggi di spazio di DFS e la completezza di BFS. È adatto per i problemi in cui la profondità della soluzione non è nota a priori.

2.9 Backtracking search

L'algoritmo backtracking search visita l'intero albero di ricerca per un dato problema fino alla profondità D , partendo dallo stato s_1 ed esplorando ricorsivamente tutti gli stati s attraverso iterazioni $\forall a \in A(s)$ per restituire il cammino migliore. Se non è possibile trovare alcun cammino in T entro una profondità D si arresta. Se la soluzione si trova oltre D oppure $|S| = \infty$ l'algoritmo non riesce a trovare una soluzione.

2.10 Riepilogo algoritmi di ricerca

Criterio	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirezionale (se applicabile)
Completo?	Sì	Sì	No	No	Sì	Sì
Costo ottimale?	Sì	Sì	No	No	Sì	Sì
Tempo	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Spazio	$O(V + E)$	$O(V + E)$	$O(V + E)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$

Figura 7: Valutazione degli algoritmi di ricerca. b è il fattore di ramificazione; m è la profondità massima dell'albero di ricerca; d è la profondità della soluzione meno profonda, o è m quando non c'è soluzione; ℓ è il limite di profondità.

2.11 Strategie di ricerca informate (euristiche)

Una strategia di ricerca informata, che utilizza suggerimenti specifici del dominio sulla posizione degli obiettivi, può trovare soluzioni in modo più efficiente rispetto a una strategia non informata. I suggerimenti si presentano sotto forma di una **funzione euristica** indicata come $h(n)$.

$h(n)$ = costo stimato del cammino meno costoso dallo stato del nodo n allo stato obiettivo.

2.12 Greedy search

L'algoritmo greedy search si basa sull'introduzione di una funzione euristica $h(s) : S \rightarrow \mathbb{R}^+$ che indica il costo ottimale atteso del cammino da s a s_G . L'algoritmo parte da s_1 ed esplora la sua frontiera, ripete questo procedimento per il nodo s tale che $h(s)$ sia minimo, fino ad arrivare alla soluzione. Se esiste almeno una soluzione, questa viene trovata ma non è detto che sia quella ottimale. Molto dipende dalla bontà dell'euristica e dal costo effettivo di a da s .

2.13 A^* search

L'algoritmo di ricerca informato più comune è A^* , una variante di best-first search che utilizza la funzione di valutazione:

$$f(n) = g(n) + h(n)$$

dove $g(n)$ è il costo del cammino dalla sorgente al nodo corrente n e $h(n)$ è il costo stimato del cammino di lunghezza minore da n fino al nodo obiettivo. Quindi risulta

$f(n)$ = la stima del costo totale di un cammino che passa per il nodo n , dalla sorgente al nodo obiettivo.

L'algoritmo A^* è completo, per far sì che sia ottimizzato in termini di costi l'euristica deve rispettare la seguente proprietà:

- **Ammissibilità:** un'euristica ammissibile non sovrastima mai il costo per raggiungere il nodo obiettivo. La stima deve essere minore uguale al costo minimo reale per raggiungere il nodo obiettivo da qualsiasi nodo.

$$h(n) \leq h^*(n)$$

La coerenza è una proprietà più forte dell'ammissibilità, se soddisfatta, implica automaticamente l'ammissibilità.

- **Coerenza:** un'euristica coerente rispetta la **disuguaglianza triangolare** ovvero, il costo per raggiungere il nodo obiettivo da n non deve essere maggiore della somma del costo per raggiungere un successore n' e la stima del costo per raggiungere l'obiettivo da n' .

$$h(n) \leq c(n, a, n') + h(n')$$

3 Classical planning

3.1 Definizione

Il classical planning consiste nel trovare una sequenza di azioni per raggiungere un obiettivo in un ambiente discreto, deterministico, statico e completamente osservabile.

3.2 STRIPS (Stanford Research Institute Problem Solver)

STRIP è un linguaggio per la rappresentazione di problemi di classical planning, un problema in STRIPS è definito come una tupla $P = (F, O, I, G)$

- F è l'insieme di tutti gli **atomi**
- O è l'insieme di tutte le **azioni**
- $I \subseteq F$ rappresenta gli atomi iniziali
- $G \subseteq F$ rappresenta gli atomi obiettivo.

Le azioni $o \in O$ sono rappresentate dalla combinazione

1. Add list $Add(o) \subseteq F$: rappresenta l'insieme delle condizioni che diventano vere dopo l'esecuzione dell'operazione.
2. Delete list $Del(o) \subseteq F$: contiene le proposizioni che diventano false dopo l'esecuzione dell'operatore.
3. Precondition list $Pre(o) \subseteq F$: rappresenta l'insieme di fatti che devono essere veri affinché l'operatore possa essere eseguito.

3.3 Struttura di un problema STRIP

Un problema STRIP $P = (F, O, I, G)$ determina un modello di stato $S(P)$ che soddisfa le seguenti condizioni:

- Gli stati $s \in S$: sono collezioni di atomi presi dall'insieme F .
- Lo stato iniziale $s_0 \in I$: rappresenta il set dei fatti veri all'inizio del problema.
- Gli stati obiettivo $s \subseteq G$: l'obiettivo è un sottoinsieme G degli atomi che devono essere veri in uno stato finale per soddisfare il problema.

- Le azioni $a \in A(s)$: le azioni sono gli operatori $o \in O$ tali che le loro precondizioni sono soddisfatte.
- Lo stato successivo $s^j = s - Del(a) + Add(a)$.
- Il costo delle azioni $c(a, s)$: ogni azione ha un costo associato, in questo caso tutte le azioni hanno lo stesso costo, cioè 1.

3.4 PDDL (Planning Domain Description Language)

PDDL è un linguaggio standard utilizzato per definire domini e problemi di pianificazione automatica. I problemi in PDDL sono specificati in due parti:

- **Dominio**: contiene schemi di azioni e atomi insieme ai tipi di argomenti.
- **Istanza**: contiene la situazione iniziale, l'obiettivo e le costanti (oggetti) di ciascun tipo.

4 Probabilistic planning

Gli agenti in ambienti reali si trovano ad affrontare dei problemi di gestione delle condizioni di incertezza, due problemi, in particolare, vengono affrontati utilizzando metodi probabilistici:

- **Percezione**: un robot attraverso i sensori potrebbe non riuscire a determinare lo stato del mondo in maniera corretta.
- **Azione**: le azioni intraprese da un robot sono spesso guidate da principi di ottimizzazione dell'utilità. Ciò significa che il robot seleziona azioni che massimizzano l'utilità attesa, calcolata in base agli stati stimati e alle incertezze associate.

La **teoria della probabilità** è lo studio di come la conoscenza influenza la credenza. La credenza in una proposizione, indicata come α , viene misurata con un numero compreso tra 0 e 1. Se la probabilità di α vale 0, allora α sarà sicuramente un evento falso, mentre se α valesse 1 l'evento sarebbe considerato sicuramente vero.

4.1 Markov localization

La localizzazione di Markov è un processo utilizzato dagli agenti per determinare il proprio stato in ambienti sconosciuti. Inizialmente l'agente non conosce il suo stato attuale, la sua conoscenza a priori dello stato è una probabilità uniforme di tutti i possibili stati. L'agente percepisce una parte dello stato attuale attraverso i sensori, utilizzando la formula di Bayes e la probabilità condizionata l'agente può aggiornare le probabilità sui vari stati possibili.

$$P(Y|X) = \frac{P(X|Y)P(X)}{P(X)} = \frac{\text{likelihood} \cdot \text{prior}}{\text{evidence}}$$

L'agente può compiere azioni che influenzano lo stato dell'ambiente. Dopo aver compiuto un'azione, l'agente aggiorna le probabilità degli stati in base alle nuove informazioni ottenute dall'azione e dai sensori.

Questo procedimento funziona sotto **l'assunzione di Markov**: *lo stato attuale dipende solamente dalla conoscenza precedente e dall'azione appena compiuta.*

4.2 Filtro ricorsivo di Bayes

Il filtro di Bayes cerca di stimare la distribuzione di probabilità dello stato attuale di un sistema, dato il suo stato precedente e le nuove osservazioni. Il sistema evolve nel tempo, e il filtro tiene traccia di come la probabilità dello stato cambia a ogni nuovo aggiornamento di dati.

4.2.1 Notazioni

- x_t : stato del sistema al tempo t .
- z_t : osservazione al tempo t .
- $P(x_t \mid z_{1:t})$: probabilità dello stato x_t dato tutte le osservazioni fino al tempo t .

4.2.2 Fase di predizione

$$P(x_t \mid z_{1:t-1}) = \int P(x_t \mid x_{t-1}) P(x_{t-1} \mid z_{1:t-1}) dx_{t-1}$$

4.2.3 Fase di aggiornamento

$$P(x_t \mid z_{1:t}) = \frac{P(z_t \mid x_t) P(x_t \mid z_{1:t-1})}{P(z_t \mid z_{1:t-1})}$$

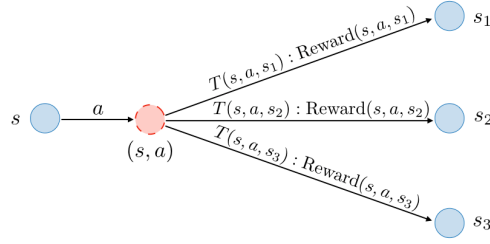
4.2.4 Passo ricorsivo

$$P(x_t \mid z_{1:t}) = \eta P(z_t \mid x_t) \int P(x_t \mid x_{t-1}) P(x_{t-1} \mid z_{1:t-1}) dx_{t-1}$$

4.3 Processo decisionale di Markov

Un processo decisionale di Markov è un modello matematico utilizzato per descrivere situazioni di decisione in ambienti che sono sia stocastici che dinamici. L'obiettivo di un processo decisionale di Markov è massimizzare le ricompense. Un problema è definito nel seguente modo:

- Un insieme di stati $s \in S$
- Un insieme di azioni $a \in A$
- Un modello di transizione $T(s, a, s')$
 - Probabilità che a da s porti a s' , cioè $P(s' \mid s, a)$
- Una funzione di ricompensa $R(s, a, s')$ per ogni transizione
- Uno stato iniziale
- Possibilmente uno stato terminale (o stato assorbente)
- Funzione di utilità che è la somma (scontata) delle ricompense



4.4 Politiche

Una soluzione al problema viene presentata specificando le azioni che un agente potrebbe compiere in ogni stato raggiungibile. Una soluzione di questo tipo è chiamata **politica** ed è indicata con il simbolo π . Una politica è una funzione che associa a ogni stato s un'azione a da intraprendere.

$$\pi : s \mapsto a$$

Una politica può essere di due tipi:

- **Deterministica:** ogni stato ha un'azione specifica associata.
- **Stocastica:** ogni stato ha una distribuzione di probabilità su possibili azioni, permettendo all'agente di scegliere in modo casuale tra le azioni disponibili.

Ogni volta che una data politica viene eseguita a partire dallo stato iniziale la natura stocastica dell'ambiente può portare a una cronologia (sequenza di stati, azioni e risultati) dell'ambiente differente. La qualità di una politica è quindi misurata dall'utilità attesa delle possibili cronologie dell'ambiente generate da quella politica.

Utility: l'utilità di un cammino $(s_0, \dots, s_k)$ è la somma scontata delle ricompense su quel cammino.

$$u(s_0, \dots, s_k) = \sum_{i=1}^k r_i \gamma^{i-1}$$

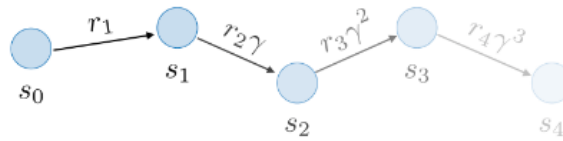


Figura 8: La figura illustra il caso in cui $K = 4$

4.5 Fattore di sconto

Il **fattore di sconto** γ è un numero compreso tra 0 e 1 e rappresenta la preferenza di un agente per le ricompense correnti rispetto a quelle future. Quando γ ha un valore vicino a 0 le ricompense nel lontano futuro sono considerate insignificanti, mentre quando γ ha un valore vicino a 1 un agente è più disposto ad attendere ricompense a lungo termine. Quando γ è esattamente 1, l'agente considera le ricompense future in modo uguale a

quelle immediate senza ridurre il loro valore nel tempo. La somma delle ricompense future viene calcolata con una serie geometrica: $r_0 + \gamma r_1 + \gamma^2 r_2$, grazie allo sconto γ il valore delle ricompense future diminuisce progressivamente, risolvendo il problema della somma infinita.



Figura 9: Processo di sconto

4.6 Equazione di Bellman

Abbiamo definito l'utilità di uno stato, $U(s)$, come la somma attesa di ricompense scontate da quel punto in poi. Da ciò ne consegue che esiste una relazione diretta tra l'utilità di uno stato e l'utilità dei suoi vicini: l'utilità di uno stato è la ricompensa attesa per la transizione successiva sommata all'utilità scontata dello stato successivo, supponendo che l'agente scelga l'azione ottimale. L'utilità di uno stato è data da:

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')]$$

4.7 Funzione di utilità Q

La funzione $Q(a, s)$ rappresenta l'utilità attesa di intraprendere una data azione in un dato stato. La funzione Q è correlata all'utilità nel seguente modo:

$$U(s) = \max_a Q(s, a)$$

Inoltre si può estrarre la politica ottima dalla funzione Q in questo modo:

$$\pi^*(s) = \arg \max_a Q(s, a)$$

È possibile sviluppare un'equazione di Bellman per le funzioni Q , notando che la ricompensa totale attesa per aver intrapreso un'azione è la sua ricompensa immediata più l'utilità scontata dello stato di esito, che a sua volta può essere espressa in termini della funzione Q :

$$\begin{aligned} Q(s, a) &= \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')] \\ &= \sum_{s'} P(s' | s, a) \left[R(s, a, s') + \gamma \max_{a'} Q(s', a') \right] \end{aligned}$$

4.8 Politica ottimale

Una politica ottimale è una politica che produce la massima utilità attesa ed è indicata con il simbolo π^* .

4.9 Value iteration

La value iteration sfrutta la **l'equazione di Bellman** e si basa sull'intuizione che il valore di uno stato può essere espresso in termini del valore degli stati successivi.

$$U_{t+1}(s) \leftarrow \max_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U_t(s')]$$

Si parte con una stima iniziale dei valori di ogni stato, per ogni stato s si aggiorna il suo valore utilizzando l'equazione di Bellman. Si applica questo procedimento all'infinito fino a raggiungere un equilibrio in cui i valori di utilità finali saranno soluzioni dell'equazione di Bellman.

4.9.1 Contrazione

Il concetto di base utilizzato per dimostrare che l'iterazione del valore converge è la nozione di **contrazione**. Una contrazione è una funzione di un argomento che, quando applicata a due input diversi a turno, produce due valori di output che sono “più vicini tra loro” almeno per un fattore costante rispetto agli input originali.

4.10 Policy iteration

L'algoritmo policy iteration è utile per trovare politiche ottime, funziona attraverso l'alternanza di due step a partire da una politica π_0 :

- **Policy evaluation:** data una policy π_i calcola $U_i = U^{\pi_i}$ l'utilità di ciascuno stato se π_i fosse eseguita.
- **Policy improvment:** aggiorna la politica migliorando le azioni in base all'utilità attuale degli stati.

L'algoritmo termina quando la fase di miglioramento della politica non produce alcun cambiamento nelle utilità. A questo punto la funzione di utilità U_i è un punto fisso del Bellman update, quindi è una soluzione alle equazioni di Bellman e π_i deve essere una politica ottimale.

4.11 SARSA λ -learning (State, Action, Reward, State, Action)

L'algoritmo SARSA learning è molto simile a quello Q -learning, tuttavia la regola di aggiornamento risulta differente in quanto SARSA si aggiorna con il valore Q dell'azione a' che viene effettivamente eseguita seguendo questa equazione:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R(s, a, s') + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Un'altra differenza consiste nel fatto che mentre Q -learning esegue il backup del valore Q dall'azione migliore in s' , SARSA attende che venga intrapresa un'azione e esegue il backup del valore Q per quell'azione.

5 Intelligenza artificiale simbolica

5.1 KRR: Knowledge Representation Reasoning

Il KRR è una disciplina che si occupa di definire dei simbolismi o dei linguaggi che permettano di formalizzare la conoscenza al fine di renderla comprensibile alle macchine, in modo tale da effettuare ragionamenti automatici ed estrapolare nuova conoscenza.

5.2 Base di conoscenza

Il componente centrale di un agente basato sulla conoscenza è la sua **base di conoscenza**, o KB. Una base di conoscenza è un insieme di frasi, ogni frase è espressa in un linguaggio chiamato linguaggio di rappresentazione della conoscenza e rappresenta alcune asserzioni sul mondo.

5.3 Differenze tra KB e database

Le KBs si differenziano dai database per vari aspetti, nelle KB la conoscenza include sia le informazioni memorizzate che le inferenze. L'organizzazione nelle KBs è ispirata a concettualizzazioni umane così come le inferenze sono modellate sul ragionamento umano.

5.4 Ragionamento e inferenze

Il ragionamento delle KB può essere classificato in diversi modi:

- **Inferenze deduttive:** si è d'accordo sul significato che viene dato ai connettivi tra le proposizioni. Un'inferenza è considerata corretta quando: se le premesse sono vere anche le conseguenze devono essere vere.
- **Inferenze del buon senso:** si è d'accordo sul significato che viene dato ai connettivi tra le proposizioni. Questo tipo di inferenza risulta corretta quando: se le premesse sono vere allora le conseguenze devono essere giustificate.
- **Logica:** si definiscono dei linguaggi che permettono di costruire un numero arbitrario di enunciati a partire da un numero di simboli anche finito. La logica indica come si debbano costruire questi linguaggi, con l'obiettivo di ottenere un linguaggio chiuso che non evolve nel tempo, ma che ha delle regole di composizione che permettono di capire se un'espressione fa parte del linguaggio oppure no. Questa interpretazione fornisce un modello matematico che stabilisce le condizioni per le quali un'inferenza è vera oppure no.

5.5 KRR e logica del primo ordine

1. **Decidibilità:** si riferisce alla possibilità di determinare, data una formula logica e un insieme di premesse, se esiste una prova finita per stabilire se la formula è vera o falsa (o se segue dalle premesse).

2. **Complessità:** per i sistemi decidibili si considera la velocità del ragionamento e la quantità di formule su cui è possibile ragionare. La complessità dipende dall'espressività del linguaggio utilizzato.
3. **Approssimazione:** si esplora la possibilità di sviluppare motori di inferenza che utilizzano regole, ovvero espressioni quasi-logiche più efficienti da gestire. Questo approccio potrebbe comportare un rilassamento della completezza delle inferenze.

5.6 Principali approcci all'intelligenza artificiali basati su KRR

1. **Declarative problem solving:** la soluzione di un problema è un'inferenza calcolata dal sistema.
2. **KRR to support different applications:** utilizzo di sistemi di rappresentazione della conoscenza per coprire un insieme di task come strumento per modellare le conoscenze di un certo dominio.

5.7 Knowledge graphs

Un knowledge graph può essere identificato come una quintupla $\langle \mathbf{E}, \mathbf{T}, \mathbf{L}, \mathbf{P}, \mathbf{A} \rangle$ in cui:

- **E:** un insieme di simboli identificativi delle entità (costanti).
- **T:** un insieme di letterali che rappresentano delle istanze di domini particolari (stringhe, numeri, ...).
- **L:** un insieme di simboli per rappresentare le classi nel knowledge graph.
- **P:** un insieme di simboli di relazione.
- **A:** un insieme di assiomi in un linguaggio.

Gli assiomi possono essere di due tipi differenti:

- **Assiomi che rappresentano i fatti:** rappresentano un'associazione di tipi all'entità e ai letterali, nei knowledge graphs tutte le istanze sono tipizzate. Oppure servono a rappresentare delle relazioni tra entità e letterali o tra coppie di entità.
- **Assiomi generali:** gli assiomi che si possono rappresentare dipendono fortemente dalla scelta del linguaggio come ad esempio RDFS e OWL.

Un requisito minimo che un knowledge graph deve soddisfare è quello di avere un sistema di tipi non piatto che permetta delle relazioni tra le classi.

5.8 Approcci alla rappresentazione di knowledge graphs

1. **Directed labelled knowledge graph:** è un grafo diretto in cui gli archi sono etichettati. Un grafo di questo tipo modella i dati utilizzando nodi e archi, dove le etichette degli archi rappresentano proprietà o relazioni.

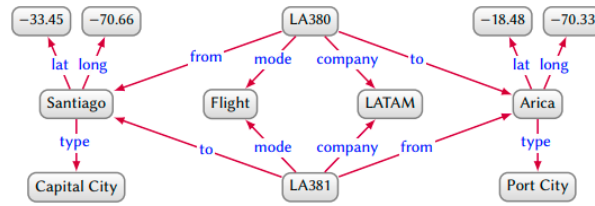


Figura 10: Directed edge-labelled graph che mostra le compagnie che offrono voli tra Santiago e Arica

2. **Property graph:** è un grafo composto da nodi, relazioni e proprietà. In questo tipo di grafo, è possibile specificare il tipo di relazione e distinguere le relazioni tra entità dagli attributi associati ai nodi e agli archi.

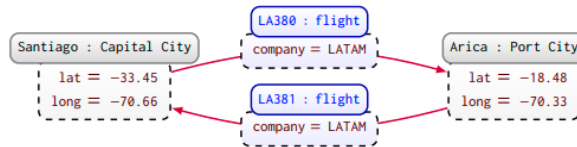


Figura 11: Property graph che mostra le compagnie che offrono voli tra Santiago e Arica

6 RDF (Resource Description Framework)

6.1 Semantic web stack

Il linguaggio RDF è stato sviluppato sia per consentire agli agenti di ragionare e scambiare dati sul web in modo interoperabile, sia per rappresentare basi di conoscenza e supportare il ragionamento su di esse. Nell'ambito del Semantic Web, è stato concepito uno stack di tecnologie per supportare questa visione.

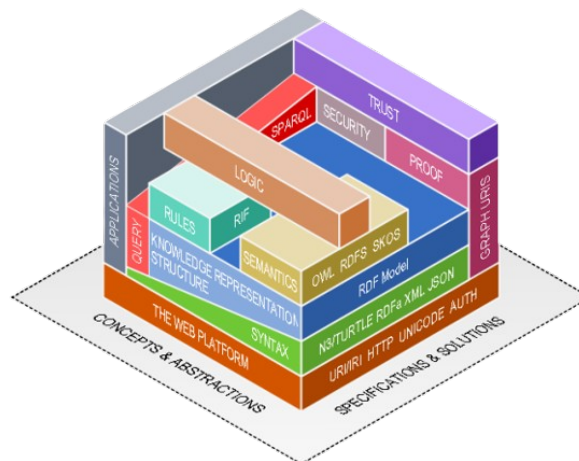


Figura 12: Semantic web stack

È organizzato in diversi livelli:

- Livello di interoperabilità.
- Livello di rappresentazione dei dati semantici sul web.
- Livello di modellazione della conoscenza (RDF).
- Livello della semantica (RDFS e OWL).

6.2 Principi base di RDF

RDF è un framework di descrizione di qualsiasi risorsa e si basa su tre principi base:

1. L'unità base di rappresentazione delle informazioni è la tripla, ogni tripla rappresenta un'affermazione, **RDF statement**, ogni affermazione è un **atomo** con un predicato binario e due costanti.
2. Ogni tripla può essere rappresentata da un grafo **diretto etichettato**, quindi è possibile avere più archi tra due nodi, anche nella stessa direzione, purché il predicato sia diverso.
3. Gli URI, **uniform resource identifiers**, sono delle stringhe di caratteri che identificano unicamente un nome o una risorsa su web. Un URI può essere di due tipi **URL**, oppure **URN**.

6.2.1 URL e URN

- **URL**: un tipo di URI che non solo specifica il tipo di risorsa ma indica anche come raggiungerla sul web.
- **URN**: un tipo di URI che usa uno schema specifico di naming URN.

6.2.2 HTTP content negotiation

È un meccanismo utilizzato nei protocolli HTTP che consente al server di fornire diverse rappresentazioni della stessa risorsa in base alle preferenze espresse dal client.

6.2.3 URIfied graph and triples

In RDF, i **prefissi** sono abbreviazioni che rendono più leggibili e gestibili gli URI utilizzati per identificare nodi e archi in un grafo. Un prefisso funge da alias per il namespace di un URI. Tra quelli più comuni ci sono:

- **dbr**: rappresenta un alias di <https://dbpedia.org/resource>
- **dbo**: rappresenta un alias di <https://dbpedia.org/ontology>
- **rdfs**: rappresenta un alias di <https://www.w3.org/2000/01/rdf-schema#>
- **rdf**: rappresenta un alias di <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

Tutte le entità provengono da **resource** mentre tutti i predicati provengono da **ontology**. **Rdf:type** è l'unico predicato nativo e viene utilizzato per catalogare le entità.

6.3 RDF Turtle/N-Triples

Ogni tripla RDF consiste di tre elementi:

1. **Soggetto:** il soggetto deve essere un IRI o un blank node non è possibile che sia un letterale.
2. **Predicato:** il predicato può essere solo un IRI.
3. **Oggetto:** gli oggetti possono essere sia IRI, sia blank node, sia letterali.

6.4 RDF linked data

Linked Data è una pratica che si basa sull'utilizzo di standard come RDF per pubblicare e collegare dati sul Web in modo strutturato e interconnesso. Il concetto chiave è rendere i dati interoperabili e collegabili attraverso diversi dataset distribuiti sul Web. I principi di linked data sono:

- **IRI:** utilizzare gli IRI come nomi delle cose.
- **Look up:** utilizzare gli IRI HTTP in modo tale che gli utenti possano ottenere delle informazioni
- **Provide useful information:** quando qualcuno cerca un IRI fornire un'informazione utili utilizzando gli standard (RDF e SPARQL).
- **Discorver:** includere link ad altri IRI in modo tale che gli utenti possano avere accesso ad altre informazioni.

6.5 Vocabolari RDF

I vocabolari RDF sono insiemi di termini utilizzati per descrivere le proprietà e le relazioni delle risorse in modo strutturato, rappresentano convenzioni condivise e accettate da una comunità per descrivere e rappresentare dati in un modo comprensibile e interoperabile. Questi vocabolari fungono da “accordi sociali” perché definiscono i termini e le relazioni che tutti i partecipanti usano per interpretare correttamente i dati in un contesto comune. **Schema.org** è l'esempio più importante e consiste in una collaborazione tra i principali motori di ricerca come Google, Microsoft e Yahoo! mirata allo sviluppo di un vocabolario comune per annotare e strutturare dati su pagine web.

7 SPARQL

SPARQL è un linguaggio di interrogazione standardizzato per il recupero e la manipolazione di dati rappresentati tramite RDF.

7.1 SPARQL basics

- **RDF graph**: un insieme di triple rdf che vengono manipolate come un grafo diretto etichettato.
- **RDF data set**: un insieme di triple RDF che comprende un grafo di default e zero o più grafi con nome.
- **SPARQL protocol client**: client HTTP che invia richieste per operazioni del protocollo SPARQL come query o aggiornamenti.
- **SPARQL protocol service**: server HTTP che gestisce le richieste per le operazioni del protocollo SPARQL.
- **SPARQL endpoint**: l'URI in cui un servizio del protocollo SPARQL ascolta le richieste dai client SPARQL.

Tripla RDF: è la struttura di base composta da soggetto, predicato e oggetto.

```
dbpedia:The_Beatles foaf:name "The Beatles" .
```

RDF triple pattern: può contenere una o più variabili. Una variabile è rappresentata con un punto di domanda davanti al nome della tripla.

```
dbpedia:The_Beatles foaf:made ?album .  
?album mo:track ?track .  
?album ?p ?o .
```

RDF quad pattern: contiene il nome del grafo sottoforma di URI o di variabile.

```
GRAPH <:g> { :s :p :o . }  
GRAPH ?g { dbpedia:The_Beatles foaf:name ?o . }
```

7.2 Pattern Matching in SPARQL

Il concetto fondamentale di SPARQL è il **pattern matching**, che consiste nell'applicare un pattern a tutto il grafo per individuare corrispondenze. Questo processo restituisce delle variabili ogni volta che il soggetto, il predicato e l'oggetto corrispondono a quelli specificati nella tripla definita. In particolare, il pattern matching identifica delle sostituzioni delle variabili che permettono di estrarre sotto-grafi del grafo originale.

7.3 Basic graph patterns

Un **basic graph pattern** è un insieme di triple che rappresentano una query su un grafo di dati. Ogni tripla è formata da variabili che possono essere mappate a nodi o relazioni nel grafo. La valutazione di un basic graph pattern su un grafo di dati consiste nel trovare tutte le possibili mappature che associano queste variabili a elementi concreti del grafo, rispettando la struttura del pattern. Queste mappature devono garantire che ogni variabile del basic graph pattern venga sostituita da un elemento del grafo di dati, generando un sotto-grafo che corrisponde al pattern originale.

- Q : rappresenta il basic graph pattern che contiene delle variabili.
- G : rappresenta un grafo di dati che contiene nodi e archi.
- μ : è una funzione parziale che associa gli elementi del pattern Q agli elementi concreti del grafo dati G convertendo le variabili astratte del pattern in nodi e archi specifici del grafo.

La valutazione di un basic graph pattern implica trovare tutte le possibili mappature μ che collegano le variabili del pattern Q ai dati nel grafo G , rispettando il vincolo che il dominio di μ (l'insieme di variabili che possono essere sostituite) deve corrispondere a tutte le variabili in Q .

7.3.1 Complex graph patterns

Un complex graph pattern è un insieme di query complesse su un grafo che si ottengono combinando o trasformando uno o più basic graph patterns utilizzando operatori algebrici relazionali. Gli operatori possono essere di due tipologie, unari e binari.

7.4 Path expression

Un'espressione di percorso è un modo di descrivere i possibili cammini in un grafo. Un'etichetta di un arco del grafo può essere considerata un'espressione di percorso, ma si possono creare espressioni di percorso più complesse utilizzando degli operatori.

- r^{-1} : indica un percorso che segue la direzione opposta rispetto a quella indicata da r .
- r^* : la chiusura di Kleene rappresenta zero o più ripetizioni di un percorso.
- $r_1 \cdot r_2$: indica la concatenazione di due percorsi, in questo caso si percorre prima r_1 e poi r_2 .
- $(r_1 \mid r_2)$ indica la disgiunzione, è possibile quindi seguire uno dei due percorsi.

7.4.1 Path evaluation

La valutazione di un'espressione di percorso su un grafo indica quali coppie di nodi possono essere collegate seguendo i cammini descritti dall'espressione.

7.5 Query components

SPARQL supporta diversi modelli di query:

- **ASK**: verifica se un query pattern ha una soluzione nel grafo e restituisce un valore booleano.
- **SELECT**: restituisce le variabili che soddisfano un pattern, insieme ai loro valori. Fornisce un insieme di risultati, dove ogni risultato è un binding (associazione) delle variabili indicate nella query.
- **CONSTRUCT**: crea un nuovo grafo RDF utilizzando un pattern di output. Il risultato è un grafo RDF generato dai dati che corrispondono al pattern.
- **DESCRIBE**: restituisce una descrizione di una risorsa RDF o di più risorse, basata sui dati disponibili.

8 RDFS (RDF Schema)

RDFS è un'estensione del modello RDF utilizzata per descrivere ontologie semplici, fornendo meccanismi per modellare classi e proprietà.

8.1 RDB Vs RDF

I database relazionali utilizzano dati strutturati con lo schema del database che definisce rigorosamente la struttura dei dati. Questo modello adotta un approccio **schema-first**, dove le tabelle vengono definite prima dell'inserimento dei dati, che devono aderire rigidamente allo schema stabilito. Al contrario, in RDF i dati sono semi-strutturati e organizzati in triple, caratterizzate da un legame debole tra dati e schema. Questo approccio è definito **data-first**, poiché i dati possono essere inseriti anche in assenza di uno schema predefinito e gli schemi non impongono vincoli stringenti sull'inserimento dei dati.

8.2 Primitive in RDFS

- **rdfs:Class**: rappresenta un insieme di risorse e consente la creazione di gerarchie tramite **rdfs:subClassOf** che modellano relazioni "is-a".
- **rdfs:Resource**: una classe che include tutte le risorse descritte in RDF.
- **rdfs:Literal**: raggruppa i valori letterali come stringhe e numeri.
- **rdfs:Datatype**: viene utilizzato per rappresentare i tipi di dato, definiti come sottoclassi di **rdfs:Literal**.
- **rdfs:subPropertyOf**: viene utilizzato per definire gerarchie tra proprietà.
- **rdfs:domain** e **rdfs:range**: sono utilizzati per specificare vincoli di dominio e range delle proprietà.

8.3 Interrogazioni e inferenze con SPARQL

SPARQL è il linguaggio standard per interrogare dati RDF, può essere integrato con inferenze RDFS per fornire risposte più ricche. Esistono due approcci per supportare le inferenze:

1. **Materializzazione**
2. **inferenza a runtime**

La **materializzazione** calcola e memorizza tutte le triple inferite prima della pubblicazione dei dati, mentre **l'inferenza a runtime** effettua il ragionamento dinamicamente. Entrambi gli approcci presentano vantaggi e svantaggi in termini di tempi di risposta e flessibilità. La materializzazione occupa più memoria ed è vincolata a quanto la conoscenza cambi rapidamente, mentre l'inferenza a runtime occupa meno spazio in memoria ma introduce degli overhead nei tempi di reazione.

8.4 Limitazioni di RDFS

RDFS presenta alcune limitazioni dovute alla natura semplice del linguaggio:

- **Impossibilità nella creazione di contraddizioni:** RDFS non permette di rilevare o gestire inconsistenze nei dati.
- **Limitazioni sulle classi disgiunte:** RDFS non consente di dichiarare classi come disgiunte.
- **Mancanza di vincoli di cardinalità:** non si possono esprimere vincoli sul numero di relazioni o proprietà che un'entità può avere.
- **Assenza di combinazioni logiche:** non si possono creare nuove classi utilizzando combinazioni logiche come unione, intersezione o complemento.
- **Restrizioni localizzate su dominio e intervallo:** RDFS applica i vincoli di dominio e intervallo in modo globale, rendendo impossibile differenziarli in base al contesto. Per esempio, non si può specificare che una proprietà abbia un certo intervallo quando utilizzata con una classe e un intervallo diverso con un'altra.

9 OWL (Ontology Web Language)

OWL è un linguaggio di ontologia che rappresenta un sottoinsieme della logica del primo ordine.

9.1 Ontologie in OWL

Le ontologie sono progettate per consentire a terzi di comprendere come è organizzata la conoscenza. In OWL, le ontologie sono definite per supportare l'interoperabilità semantica, ovvero per garantire che possano essere comprese e riutilizzate in diversi contesti. Questo significa che un'ontologia non viene ricreata ogni volta che se ne ha bisogno; al contrario, è possibile riutilizzare ontologie esistenti oppure importarne altre, sfruttando la modularità offerta da OWL.

9.2 Struttura e profili di OWL 2

OWL 2, evoluzione del linguaggio, è diviso in profili progettati per specifiche esigenze:

- **OWL 2 EL**: ideale per classificazioni di base con ragionamento in tempo polinomiale.
- **OWL 2 QL**: ottimizzato per essere tradotto in query di database relazionali.
- **OWL 2 RL**: progettato per l'implementazione efficiente in sistemi basati su regole.

9.3 Class axioms

Gli assiomi dichiarano affermazioni generali sui concetti che vengono utilizzati nell'inferenza logica, in OWL ci sono due costrutti per indicare che due classi sono equivalenti oppure disgiunte.

- **Equivalent relationship**: le classi hanno gli stessi individui.

`owl:equivalentClass`

- **Disjointness**: le classi non hanno gli stessi individui.

`owl:disjointWith`

9.4 Axioms about individuals

Le risorse sono collegate alle classi tramite la proprietà `rdf:type`.

- È possibile dichiarare che due risorse sono uguali utilizzando la proprietà

`owl:sameAs`

- È possibile dichiarare che due risorse sono diverse utilizzando la proprietà

`owl:differentFrom`

9.5 OWL properties

OWL distingue due tipi di proprietà

- **OWL ObjectProperties**: risorse come valore, collegano risorse a risorse.
- **OWL DatatypeProperties**: letterali come valore, collegano risorse a letterali.

9.6 Property axioms

OWL consente la definizione di caratteristiche di proprietà per dedurre nuovi fatti relativi alle istanze e alle loro proprietà.

- **Simmetria**

```
<p rdf:type owl:SymmetricProperty>
  <x p y> iff <y p x>
```

- **Inversa**

```
<p owl:inverseOf q>
  <x p y> iff <y q x>
```

- **Transitività**

```
<p rdf:type owl:TransitiveProperty>
  If (<x p y> and <y p z>) then <x p z>
```

- **Essere funzionale**

Una funzione è una relazione tale per cui un argomento, il primo elemento, è associato al più a un valore. Una relazione funzionale è una relazione che si comporta come una funzione.

```
<p rdf:type owl:FunctionalProperty>
  If (<x p y> and <x p z>) then y owl:sameAs z
```

- **Inversamente funzionale**

Una relazione inversamente funzionale è una relazione il cui inverso si comporta come una funzione.

```
<p rdf:type owl:InverseFunctionalProperty>
  If (<x p z> and <y p z>) then x owl:sameAs y
```

9.7 Struttura di una classe OWL

Le classi in OWL sono organizzate in gerarchie che costituiscono un **poset** (partially ordered set) di classi. La relazione di sottoclasse definisce questa gerarchia in modo transitivo e riflessivo, creando un ordinamento parziale tra le classi. A differenza di una semplice struttura ad albero, OWL supporta l'ereditarietà multipla, consentendo a una classe di avere più classi madri. Di conseguenza, la struttura complessiva delle classi non è una semplice gerarchia, ma assume la forma di un **reticolo**. Per completare questa struttura reticolare, OWL introduce due concetti speciali: `owl:Thing`, la super-classe di tutte le classi, che rappresenta il vertice superiore del reticolo e `owl:Nothing`, la classe vuota che costituisce il vertice inferiore.

9.8 Completezza delle knowledge bases

Le basi di consocenza sono intrinsecamente incomplete e utilizzano di default la open word assumption.

- **Open word assumption:** implica che se non si hanno prove che un'informazione faccia parte della knowledge base, non si può assumere che sia falsa.
- **Close word assumption:** prevede che tutto ciò che non è provato essere vero si assume sia falso.
- **Unique name assumption:** è un'assunzione per cui nomi differenti si riferiscono a entità differenti.

9.9 Description logics

OWL 2 si basa sulla logica descrittiva SROIQ, questa base garantisce semantiche ben definite e formalmente comprensibili, supportando algoritmi di ragionamento ottimizzati e sistemi implementati. I concetti chiave nelle DL includono:

- **Concetti (classi)**
- **Ruoli (proprietà/relazioni)**
- **Individui (istanze)**

9.10 Costruttori

In OWL 2, i costruttori di concetti vengono utilizzati per definire classi e relazioni in modo dettagliato e flessibile, consentendo di modellare conoscenze complesse. I concetti possono essere combinati e manipolati usando operazioni logiche e restrizioni, in particolare:

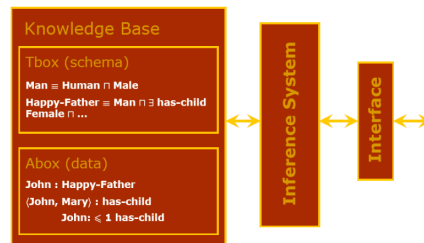
- **Intersezione ($\sqcap$):** permette di definire un nuovo concetto come l'insieme degli elementi appartenenti contemporaneamente a due o più classi.
- **Unione ($\sqcup$):** rappresenta l'insieme degli elementi che appartengono ad almeno una delle classi specificate.
- **Complemento ($\neg$):** definisce ciò che non appartiene a un determinato concetto.

OWL 2 consente anche di specificare vincoli attraverso restrizioni sui ruoli, ovvero sulle proprietà che collegano individui a classi.

- **Restrizioni universali ($\forall R.C$):** dichiarano che per un determinato ruolo, ogni elemento correlato appartiene a una certa classe. La restrizione **allValuesFrom** implica che, per una determinata proprietà di una classe, tutti i valori correlati debbano appartenere a una specifica classe.
- **Restrizioni esistenziali ($\exists E.C$):** richiedono che esista almeno un elemento correlato appartenente a una determinata classe. La restrizione **someValuesFrom**, invece, richiede che almeno un valore di una proprietà appartenga a una specifica classe.

OWL 2 implementa anche una gestione delle cardinalità che specifica il numero minimo o massimo di relazioni che un individuo può avere con altri. Oltre ai costruttori per definire concetti, OWL 2 utilizza gli assiomi per formalizzare la conoscenza all'interno dell'ontologia, questi si suddividono in due categorie principali:

1. **Abox**: contiene informazioni su individui specifici, come l'appartenenza a una classe.
2. **Tbox**: definisce relazioni tra classi, come inclusioni o equivalenze.
 - **Inclusione concettuale**: $C \subseteq D$ indica che tutti gli individui di C appartengono anche a D .
 - **Equivalenza**: $C \equiv D$ indica che C e D sono concetti con la stessa interpretazione.
3. **Rbox**: definisce le proprietà e le loro relazioni, come inclusione e disgiunzione tra ruoli.



9.11 Sintassi e semantica

In tutte le logiche la **sintassi** è definita in modo induttivo, si parte dagli elementi base per poi stabilire regole di composizione che permettono di costruire frasi arbitrariamente complesse. In questo contesto, gli elementi fondamentali sono i **concetti**, i **ruoli** e gli **assiomi**. La **semantica** viene definita attraverso una funzione di interpretazione, che specifica come devono essere interpretati i costrutti del linguaggio. Tuttavia, una logica non determina direttamente ciò che è vero o falso, ma identifica quali **inferenze** sono **valide**, ovvero quelle che risultano vere in tutte le interpretazioni che soddisfano le premesse. Prima di interpretare le formule di una logica è necessario definire come interpretare i **termini** e i **predicati** utilizzati nel linguaggio. Un termine può rappresentare un individuo, mentre un predicato può rappresentare una relazione tra individui. Un'interpretazione di un termine è un individuo nell'insieme del dominio, un'interpretazione di un predicato, se è unario è un insieme di individui nel dominio, se è binario è un insieme di coppie nel dominio, se è ternario è un insieme di triple nel dominio e così via.

9.11.1 Interpretazione

Un'interpretazione è una coppia costituita da un dominio di interpretazione e da una funzione di interpretazione. La funzione di interpretazione è definita ricorsivamente e specifica come interpretare i concetti atomici, i nomi degli individui e i ruoli del linguaggio, nonché i concetti complessi, i ruoli complessi e le proprietà in base ai costruttori utilizzati. La base della funzione di interpretazione è la seguente:

- Ogni concetto atomico viene interpretato come un sottoinsieme del dominio di interpretazione;
- Ogni ruolo atomico viene interpretato come una relazione binaria (sottoinsieme del prodotto cartesiano) sul dominio di interpretazione;
- Ogni nome di individuo viene interpretato come un elemento del dominio di interpretazione.

10 Intelligenza artificiale sub-simbolica

10.1 Introduzione all'apprendimento automatico

L'apprendimento automatico si distingue in due principali categorie: apprendimento **supervisionato** e apprendimento non **supervisionato**.

- **Supervised learning**: consiste nello sviluppo di un modello predittivo basato su coppie di input-output. Ad esempio, per analizzare immagini non note, è necessario disporre di un dataset etichettato contenente immagini già classificate con le rispettive categorie desiderate. Questo approccio si divide ulteriormente in due sotto-categorie:
 - **Classificazione**: utilizzata per assegnare un'etichetta ad un dato.
 - **Regressione**: utilizzata per prevedere valori continui.
- **Unsupervised learning**: è una tecnica di machine learning in cui un modello apprende esclusivamente dai dati di input senza disporre di output etichettati. L'obiettivo principale è identificare strutture o schemi nascosti nei dati. Un esempio comune è il clustering, una tecnica che suddivide il dataset in gruppi significativi in base a caratteristiche simili.
- **self-supervised learning**: è una tecnica non supervisionata che utilizza modelli progettati per l'apprendimento supervisionato (come le reti neurali), ma senza necessità di etichettatura manuale.

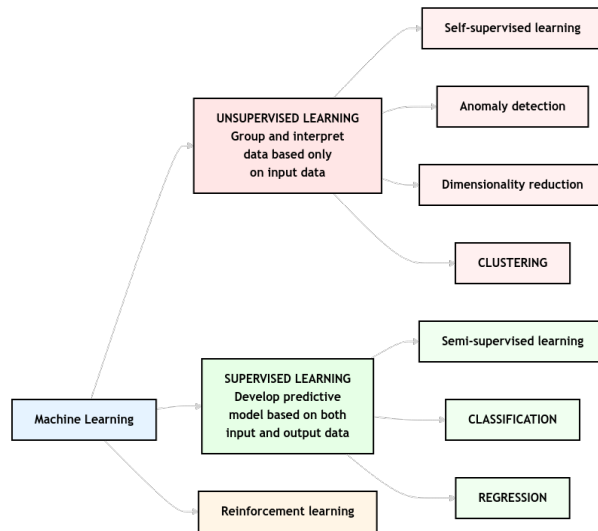


Figura 13: Diverse tipologie di machine learning

- **Semi-supervised learning:** è una tecnica che utilizza dati di addestramento parzialmente etichettati combinando algoritmi sia supervisionati che non supervisionati.
- **Reinforcement learning:** è una tecnica che prevede che un agente impari interagendo con l'ambiente ricevendo ricompense o punizioni per le sue azioni.

10.2 Machine learning Vs traditional programming

La **programmazione** si articola in diverse fasi: l'analisi del problema, la scrittura del codice e la verifica attraverso il testing. Se il test ha esito positivo si procede con il deployment, in caso contrario si torna a lavorare sull'implementazione. Il programma riceverà un input e restituirà un output.

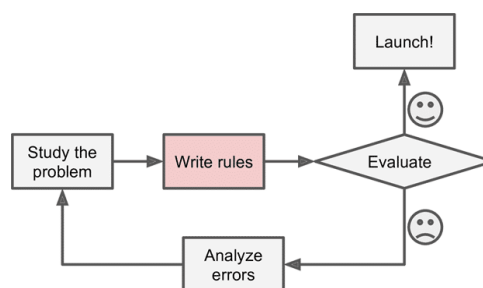
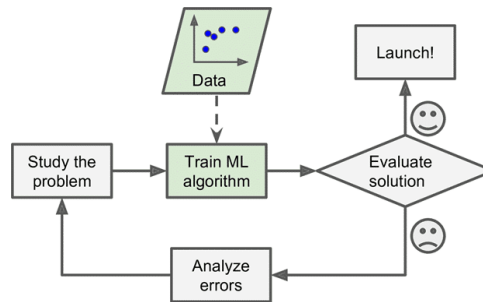


Figura 14: Step della programmazione tradizionale

Un **sistema di apprendimento automatico** prevede una fase di analisi del problema in cui si cerca di comprendere quali dati siano necessari, come siano strutturati e quanti siano disponibili. Successivamente, si seleziona un algoritmo di apprendimento automatico che viene utilizzato per generare un modello. A differenza della programmazione tradizionale l'output di un sistema di apprendimento supervisionato non è una risposta diretta all'input, ma un programma.



10.3 Supervised machine learning

L'apprendimento supervisionato è una tecnica di apprendimento automatico in cui i dati di addestramento forniti all'algoritmo includono le soluzioni desiderate, chiamate **labels** (etichette). Queste etichette possono rappresentare:

- **Categorie:** ad esempio SPAM o NO SPAM nei problemi di classificazione.

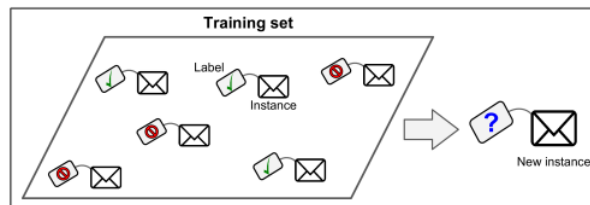


Figura 15: Esempio di training set per classificare le mail SPAM / NON SPAM

- **Valori numerici:** utilizzati nei problemi di regressione per apprendere una funzione incognita.

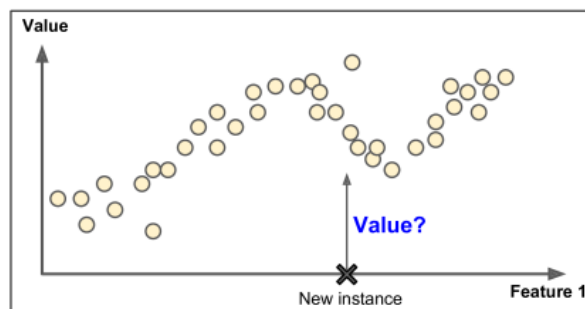


Figura 16: Regressione

10.4 Unsupervised machine learning

Nell'apprendimento non supervisionato i dati di addestramento non sono etichettati, questo approccio si concentra sull'identificazione di schemi all'interno dei dati. Tra le tecniche di apprendimento non-supervisionato ci sono:

- **Clustering:** algoritmi che raggruppano istanze simili all'interno del dataset in base alle loro caratteristiche.

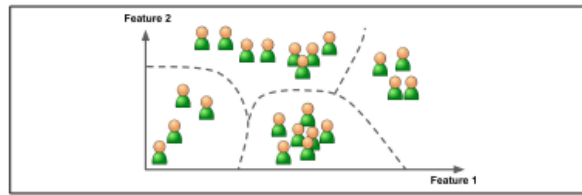


Figura 17: Clustering

- **Dimensionality reduction:** è una tecnica utilizzata per ridurre il numero di variabili in un dataset ad alta dimensionalità, preservando il più possibile le informazioni rilevanti, al fine di semplificare l'analisi, migliorare l'efficienza computazionale e facilitare la visualizzazione.
- **Anomaly detection:** è una tecnica utilizzata per identificare dati che si discostano significativamente da un insieme di dati considerati normali, rilevando così anomalie o comportamenti atipici.

10.5 Batch Vs online learning

- L'apprendimento di tipo **batch** prevede che il modello venga addestrato utilizzando l'intero dataset disponibile. Questo approccio richiede molte risorse, sia temporali, sia computazionali rendendolo adatto ad un addestramento offline.
- L'apprendimento **online**, invece, prevede che il modello venga addestrato in modo incrementale, elaborando i dati in piccoli gruppi **mini-batch** o anche singolarmente. Ciò permette un adattamento più rapido al cambiamento dei dati.

10.6 Instance-based learning

Nell'ambito del machine learning, non è sempre necessario intraprendere un processo di addestramento esplicito del modello. Nel caso dell'**instance-based learning**, il sistema apprende direttamente dagli esempi memorizzati e generalizza ai nuovi casi confrontandoli con quelli già noti utilizzando una misura di similarità. Ad esempio, nella creazione di un filtro antispam, il sistema potrebbe segnalare come spam anche email simili a quelle già identificate, basandosi su una misura di similarità tra i contenuti delle email, come il conteggio delle parole in comune con quelle classificate come spam.

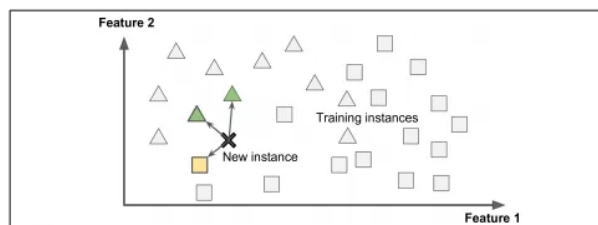


Figura 18: Instance-based learning

11 Supervised Learning

11.1 Concetti Base

Un dataset è costituito da un insieme di record (detti anche esempi, istanze o casi), descritti da k attributi $A_1, A_2, \dots, A_k$, e associati a una classe predefinita per ciascun esempio. L'obiettivo è apprendere un modello di classificazione dai dati che possa essere utilizzato per prevedere le classi di nuovi casi futuri. Se l'output non appartiene a un insieme finito di classi discrete, ma rappresenta un valore numerico si parla di regressione anziché di classificazione.

ID	Age	Has job	Own house	Credit rating	Class
1	young	false	false	fair	No
2	young	false	false	good	No
3	young	true	false	good	Yes
4	young	true	true	fair	Yes
5	young	false	false	fair	No
6	middle	false	false	fair	No
7	middle	false	false	good	No
8	middle	true	true	good	Yes
9	middle	false	true	excellent	Yes
10	middle	false	true	excellent	Yes
11	old	false	true	excellent	Yes
12	old	false	true	good	Yes
13	old	true	false	good	Yes
14	old	true	false	excellent	Yes
15	old	false	false	fair	No

Figura 19: Esempio di dataset

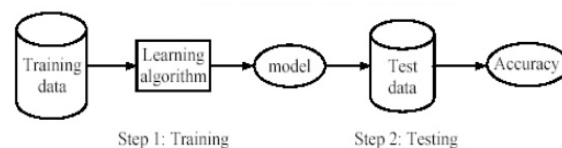
11.2 Supervised learning process

Il processo di apprendimento del modello prevede due fasi:

1. **Learning (training)**: viene utilizzato un algoritmo di apprendimento per addestrare il modello con dei dati di addestramento.
2. **Testing**: il modello costruito viene testato tramite l'utilizzo di dati “non visti” cioè dati che non sono stati utilizzati per addestrare il modello in modo tale da verificarne l'accuratezza.

L'accuratezza di un modello è data dalla seguente formula:

$$\text{Accuracy} = \frac{\text{Number of correct classifications}}{\text{Total number of test cases}}$$

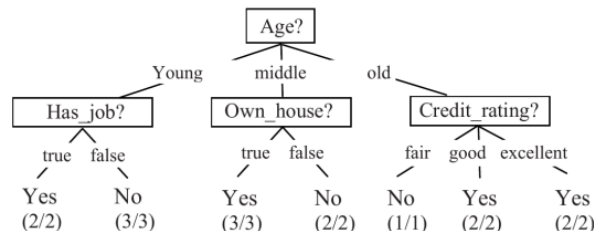


11.2.1 Assunzione fondamentale nell'apprendimento

Una delle assunzioni fondamentali nell'ambito del machine learning è che la distribuzione degli esempi di training sia identica alla distribuzione degli esempi di test, inclusi i casi futuri non ancora osservati. Tuttavia, nella pratica questa assunzione viene spesso violata e in caso di violazioni significative il risultato è una scarsa accuratezza nella classificazione.

11.3 Decision tree induction

L'apprendimento basato sugli alberi decisionali è una tecnica competitiva soprattutto su dati strutturati in forma tabellare. La tecnica prevede la costruzione di un albero i cui nodi conterranno i test sugli attributi di un nuovo individuo, navigando l'albero dalla radice ad una foglia sarà possibile valutare gli attributi dell'individuo e assegnarli ad una determinata classe.



Gli alberi di decisione non sono unici, trovare l'albero decisionale migliore corrisponde a un problema np completo. Tutti gli algoritmi per costruire alberi di decisione sono euristici, ovvero non garantiscono l'ottimalizza ma hanno buone proprietà computazionali. Inoltre un albero di decisione può essere convertito in un insieme di regole e ogni regola può essere associata a quanti elementi del dataset supportano una determinata regola.

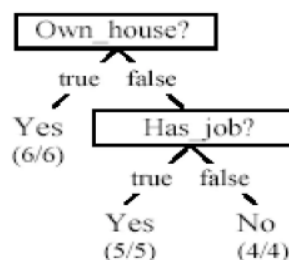


Figura 20: Albero decisionale accurato ma ridotto

11.4 Algoritmo per l'apprendimento dell'albero decisionale

L'algoritmo di base per l'apprendimento degli alberi decisionali segue un approccio **divide-et-impera** di tipo **greedy**. La costruzione dell'albero è ricorsiva e procede dall'alto verso il basso, partendo con tutti gli esempi di training posizionati nel nodo radice. Questi esempi vengono poi suddivisi in modo ricorsivo sulla base degli attributi selezionati, determinati da una funzione di impurità come l'information gain. Il partizionamento si ferma al verificarsi di specifiche condizioni:

- Tutti gli esempi in un dato nodo appartengono alla stessa classe.
- Non ci sono più attributi disponibili per ulteriori suddivisioni (in questo caso si assegna la classe maggioritaria al nodo foglia).
- Non rimangono più esempi da classificare.

Algorithm 1 decisionTree(D, A, T)

```
1: if  $D$  contains only training examples of the same class  $c_j \in C$  then
2:   make  $T$  a leaf node labeled with class  $c_j$ .
3: else if  $A = \emptyset$  then
4:   make  $T$  a leaf node labeled with  $c_j$ , which is the most frequent class in  $D$ .
5: else
6:   //  $D$  contains examples belonging to a mixture of classes. We select a single
7:   // attribute to partition  $D$  into subsets so that each subset is purer
8:    $\rho_0 = \text{impurityEval-1}(D)$ 
9:   for each attribute  $A_i \in \{A_1, A_2, \dots, A_k\}$  do
10:     $\rho_i = \text{impurityEval-2}(A_i, D)$ 
11:   end for
12:   Select  $A_g \in \{A_1, A_2, \dots, A_k\}$  that gives the biggest impurity reduction, computed
   using  $\rho_0 - \rho_i$ 
13:   if  $\rho_0 - \rho_g < \text{threshold}$  then
14:     //  $A_g$  does not significantly reduce impurity  $\rho_0$ 
15:     make  $T$  a leaf node labeled with  $c_j$ , the most frequent class in  $D$ .
16:   else
17:     //  $A_g$  is able to reduce impurity  $\rho_0$ 
18:     Make  $T$  a decision node on  $A_g$ 
19:     Let the possible values of  $A_g$  be  $v_1, v_2, \dots, v_m$ .
20:     Partition  $D$  into  $m$  disjoint subsets  $D_1, D_2, \dots, D_m$  based on the  $m$  values of  $A_g$ .
21:     for each  $D_j$  in  $\{D_1, D_2, \dots, D_m\}$  do
22:       if  $D_j \neq \emptyset$  then
23:         create a branch (edge) node  $T_j$  for  $v_j$  as a child node of  $T$ 
24:         decisionTree( $D_j, A - \{A_g\}, T_j$ ) //  $A_g$  is removed
25:       end if
26:     end for
27:   end if
28: end if
```

11.4.1 Come scegliere un attributo per partizionare il dataset

La chiave per la scelta dell'attributo sul quale costruire il sotto-ramo è ridurre l'**impurità** nel dataset considerato. Un sottoinsieme è considerato puro se tutte le istanze appartengono alla stessa classe.

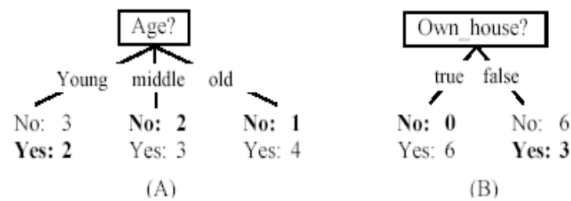


Figura 21: La soluzione B è quella migliore

11.5 Teoria dell'informazione

La teoria dell'informazione quantifica matematicamente quanto sia informativo un evento. Il principio base è che un'informazione è tanto più preziosa quanto più è imprevedibile. Per esempio, il risultato del lancio di una moneta truccata (99%) contiene meno informazione rispetto al lancio di una moneta equa (50-50), poiché nel primo caso il risultato è più prevedibile. Questa quantità di informazione viene misurata in bit, dove un bit rappresenta l'informazione contenuta in una risposta sì/no completamente imprevedibile.

11.5.1 Entropia

Il concetto di entropia indica il valore di impurità all'interno di un dataset e si esprime attraverso la seguente formula:

$$\text{entropy}(D) = - \sum_{j=1}^{|C|} \text{Pr}(c_j) \log_2 \text{Pr}(c_j)$$

Dove $\text{Pr}(c_j)$ è la probabilità della classe c_j in un dataset D .

11.5.2 Information gain

Dato un insieme D di esempi prima viene calcolata la sua entropia:

$$\text{entropy}(D) = - \sum_{j=1}^{|C|} \text{Pr}(c_j) \log_2 \text{Pr}(c_j)$$

Se scegliamo l'attributo A_i con v valori come radice dell'albero corrente, questo partiziona D in v sottoinsiemi $D_1, D_2, \dots, D_v$. L'entropia attesa se A_i viene usato come radice è:

$$\text{entropy}_{A_i}(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times \text{entropy}(D_j)$$

Il guadagno di informazione ottenuto selezionando l'attributo A_i per partizionare i dati è:

$$\text{gain}(D, A_i) = \text{entropy}(D) - \text{entropy}_{A_i}(D)$$

Scegliamo l'attributo con il guadagno più alto per partizionare l'albero in esame.

11.5.3 Gestione degli attributi continui

Per gestire un attributo continuo si sceglie una soglia per dividere il dominio dell'attributo durante il processo di branching. Per individuare la soglia, si ordinano in modo crescente tutti i valori dell'attributo presenti nel dataset e si identifica il valore che, se utilizzato come soglia, massimizza il guadagno informativo.

11.6 Overfitting negli alberi decisionali

Un algoritmo di apprendimento automatico deve essere in grado di generalizzare i dati, ossia di effettuare previsioni accurate anche su dati mai visti prima. Quando il modello non generalizza ma si limita a memorizzare il dataset di addestramento, si verifica il fenomeno dell'**overfitting**. Questo problema si manifesta quando il modello ottiene ottimi risultati sui dati di training ma ha prestazioni scarse su dati nuovi. Negli alberi decisionali, un sintomo di overfitting è indicato dalla presenza di un albero eccessivamente profondo. Per evitare l'overfitting negli alberi decisionali, si possono adottare due tecniche principali:

- **Pre-pruning:** questa tecnica interrompe la costruzione di un sotto-albero quando il guadagno informativo è minimo evitando di procedere con suddivisioni non significative.
- **Post-pruning:** in questo approccio l'albero viene costruito completamente sulla base dei dati per poi essere semplificato utilizzando tecniche statistiche.

11.7 Valutazione dei classificatori

Esistono diverse metriche che vengono utilizzate per valutare le prestazioni di un modello tra cui:

- **Accuratezza:**

$$\text{Accuracy} = \frac{\text{Number of correct classifications}}{\text{Total number of test cases}}$$

- **Efficienza**

- Tempo necessario per costruire il modello.
- Tempo necessario per utilizzare il modello.

- **Robustezza:** indica il modo in cui il modello gestisce il rumore e i valori mancanti nei dati.
- **Scalabilità:** indica l'efficienza del modello quando i dati sono di grandi dimensioni.
- **Interpretabilità:** indica quanto il modello sia comprensibile e in grado di fornire intuizioni.
- **Compattezza del modello:** indica le dimensioni del modello come il numero di regole o la grandezza dell'albero decisionale.

12 Metriche di valutazione

Come già affermato in precedenza, i dati utilizzati nel processo di addestramento del modello non dovrebbero essere impiegati per valutarne l'accuratezza. Pertanto, è possibile suddividere il dataset di partenza in due sottoinsiemi distinti tramite il metodo **Holdout set** che prevede:

- **Sottoinsieme di addestramento:** D_{train} contiene i dati per addestrare il modello.
- **Sottoinsieme di test:** D_{test} contiene i dati per effettuare la procedura di testing sul modello.

Solitamente questo metodo è utilizzato quando il dataset è abbastanza grande in modo tale che circa l'80% dei dati possa essere utilizzato per l'addestramento e il restante 20% per la fase di testing, così da ottenere un modello affidabile e rappresentativo.

12.1 N-fold cross-validation

Quando il dataset non è sufficientemente grande viene suddiviso attraverso il metodo della **n-fold cross-validation** in n sottoinsiemi di egual grandezza. Ogni sottoinsieme viene, a turno, utilizzato come set di test mentre i restanti $n - 1$ sottoinsiemi vengono combinati per costituire il set di addestramento. Questo procedimento viene ripetuto n volte, producendo n misure di accuratezza. L'accuratezza finale del modello è calcolata come la media delle n accuratezze ottenute. Tra le varianti più comuni di questo metodo vi sono la 10-fold e la 5-fold cross-validation in cui il dataset viene suddiviso rispettivamente in 10 e 5 sottoinsiemi.

12.1.1 Leave-one-out cross-validation

Si tratta di un altro approccio di cross-validation che prevede l'utilizzo dell'intero dataset nel caso in cui questo sia molto piccolo. Ogni sottoinsieme della cross-validation ha un solo esempio di test mentre la restante parte del dataset viene utilizzata per addestrare il modello.

12.2 Validation set

Il validation set è un sottoinsieme di dati utilizzato nei processi di apprendimento automatico. In molti casi i dati vengono suddivisi in tre parti:

1. **Training set**: un set di addestramento
2. **Validation set**: un set di validazione
3. **Test set**: un set di test

Il **validation set** è impiegato per stimare e ottimizzare i parametri del modello, i valori dei parametri che garantiscono la migliore accuratezza sul validation set vengono scelti come definitivi. Un'alternativa al validation set per stimare i parametri può essere la cross-validation.

12.3 Misure di classificazione

L'accuratezza è un indicatore veritiero, tuttavia pur essendo un numero singolo è estremamente aggregata e non distingue tra casistiche differenti. Alcune classi, per esempio, potrebbero essere più sensibili all'errore, inoltre errori diversi hanno conseguenze diverse, pertanto potrebbe essere necessario distinguere il tipo di errore. In una classe binaria si può distinguere tra **classificazione positiva** e **classificazione negativa**.

12.4 Metriche Precision e Recall

In una classificazione binaria si utilizza una **matrice di confusione** che consiste in una tabella necessaria a valutare le prestazioni di un classificatore in un problema di classificazione supervisionata.

	Class 1 Predicted	Class 2 Predicted
Class 1 actual	TP	FN
Class 2 actual	FP	TN

Details of the confusion matrix:

- Positive (P): Actually, positive.
- Negative (N): Actually, it is not positive.
- True Positive (TP): Actually positive and predicted as positive.
- False Negative (FN): Actually positive but predicted as negative.
- True Negative (TN): Actually negative and predicted as negative.
- False Positive (FP): Actually negative but predicted as positive.

Figura 22: Matrice di confusione

12.4.1 Precision

La metrica **precision** rappresenta il rapporto tra il numero di esempi positivi classificati correttamente e il numero totale di esempi positivi. Precision fa riferimento all'autorevolezza di un classificatore stante una classificazione positiva.

$$p = \frac{TP}{TP+FP}$$

12.4.2 Recall

La metrica **recall** rappresenta il rapporto tra il numero di esempi positivi classificati correttamente e il numero totale di esempi presenti realmente nel test set. Recall si riferisce a quanto il classificatore copra accuratamente l'insieme di tutti i positivi.

$$r = \frac{TP}{TP+FN}$$

Nota Bene: Precision e recall si concentrano solamente su una classe positiva

12.5 F_1 -Score

La metrica f_1 - score è una misura di valutazione usata per combinare due metriche fondamentali, precision e recall in un unico valore, rendendo più semplice confrontare le prestazioni di due classificatori. f_1 - score essendo una media armonica dà maggiore peso al valore più piccolo tra precision e recall. Questo implica che entrambi devono essere alti affinché f_1 - score sia elevato.

$$F_1 = \frac{2pr}{p+r}$$

- Se una delle due metriche è bassa, anche l' F_1 - score sarà basso, indipendentemente dall'altra.
- L' F_1 - score sarà elevato solo se entrambi precision e recall sono alti.

12.6 Receiver operating characteristic curve (ROC curve)

La ROC curve è una rappresentazione grafica che illustra la relazione tra il tasso di veri positivi (**TPR**) e il tasso di falsi positivi (**FPR**), al variare della soglia di classificazione. In contesti dove i dati sono sbilanciati è molto difficile utilizzare una classificazione binaria, risulta più utile utilizzare un regressore che fornisce una probabilità di appartenenza di

un'istanza a una certa classe. In contesti in cui i dati sono sbilanciati, la classificazione binaria diretta può risultare poco efficace. In questi casi, è preferibile utilizzare un modello che fornisca un punteggio continuo o una probabilità di appartenenza di un'istanza a una determinata classe $\Pr(+|X)$, ossia la probabilità di un risultato positivo dato l'input x . Tale punteggio può essere utilizzato sia per applicare una soglia appropriata convertendo la probabilità in classificazioni binarie, sia per lavorare direttamente sul ranking ordinando le istanze in base ai punteggi stimati.

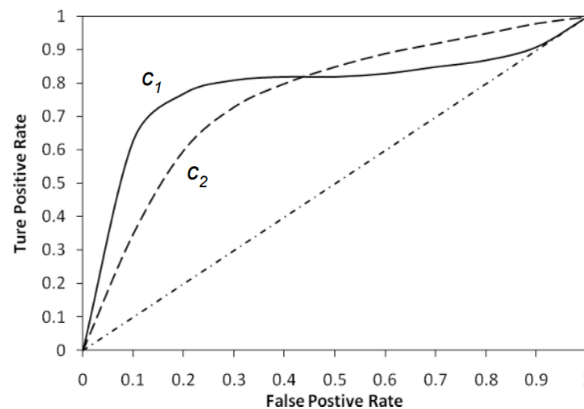


Figura 23: ROC curves per due classificatori (C_1 e C_2) sullo stesso dataset

12.6.1 Area sotto la curva (AUC)

Quando si confrontano due classificatori C_1 e C_2 , le loro performance possono variare a seconda della soglia considerata. Per superare questa limitazione, si utilizza l'**AUC** che rappresenta l'area sotto la ROC curve. L'AUC fornisce una misura complessiva della capacità del classificatore di distinguere tra le due classi, indipendentemente dalla soglia scelta. Se l'AUC del classificatore C_1 è maggiore di quella del classificatore C_2 , C_1 è un classificatore migliore di C_2 viceversa altrimenti. Un classificatore perfetto avrà un valore di AUC pari a 1, poiché sarà sempre in grado di distinguere correttamente le istanze positive da quelle negative. Al contrario, un classificatore che opera in modo casuale avrà un valore di AUC pari a 0.5.

12.7 k-Nearest Neighbor Classification (kNN)

kNN è un algoritmo di apprendimento supervisionato che non richiede una fase esplicita di addestramento di un modello. Funziona classificando un'istanza di test d basandosi sulle k istanze più vicine nello spazio dei dati.

Funzionamento

- Per un'istanza test d , si trovano i k vicini più prossimi;
- Si contano gli n esempi tra k vicini che appartengono a ciascuna classe c_j ;
- Si stima $\Pr(c_j|d)$ ovvero la possibilità che l'istanza d appartenga alla classe c_j ;

- Si assegna d alla classe con la frequenza maggiore, quella più comune tra i k vicini.

Per un'istanza test d , si trovano i k vicini più prossimi, si contano gli n esempi tra k vicini che appartengono a ciascuna classe c_j .

12.7.1 Caratteristiche di kNN

- kNN non prevede la costruzione di un modello, ma lavora direttamente sui dati di training.
- Il tempo per classificare un'istanza test cresce linearmente con il numero di esempi di training.
- k viene scelto empiricamente, spesso utilizzando il metodo di cross-validation.
- La funzione distanza è estremamente importante, ma dipende dall'applicazione.

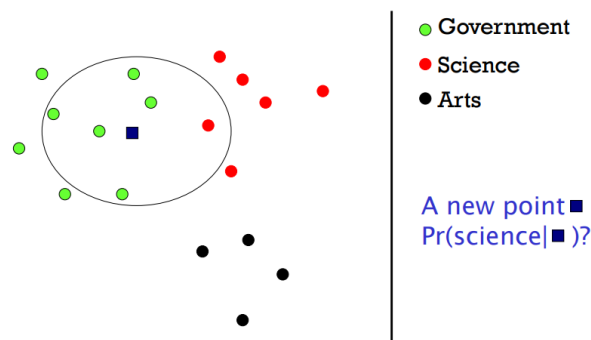


Figura 24: Esempio: $k = 6$ (6NN)

12.8 Ensemble methods

L'idea di **metodo ensemble** consiste nel combinare più classificatori con l'obiettivo di ottenere un classificatore complessivo migliore, che superi le prestazioni dei singoli componenti:

1. **Bagging (bootstrap aggregating)**: è una tecnica di ensemble che utilizza un dataset D contenente m esempi di addestramento. Vengono creati k campioni bootstrap $S[1], S[2], \dots, S[k]$, ciascuno generato selezionando m esempi dal dataset D con rimpiazzamento, il che significa che lo stesso esempio può comparire più volte in un singolo campione. Successivamente, vengono addestrati k classificatori indipendenti utilizzando lo stesso algoritmo di apprendimento, uno per ciascun campione bootstrap. Le predizioni finali sono ottenute combinando i risultati di k classificatori. Per un problema di regressione si fa la media degli output, per un problema di classificazione si sceglie la classe con la più alta maggioranza di voti.

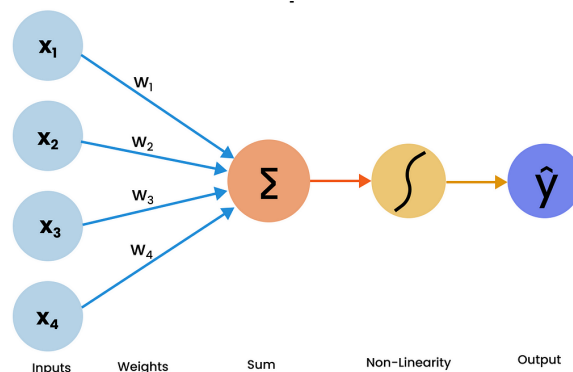
2. **Boosting (AdaBoost)**: durante la fase di training viene prodotta una sequenza di classificatori che utilizza lo stesso base learner. Ogni classificatore dipende dai risultati del precedente e si focalizza sugli errori commessi. Gli esempi che sono stati classificati in modo errato dai classificatori precedenti ricevono pesi più alti affinché il classificatore successivo presti maggiore attenzione a tali errori. Nella fase di testing i risultati dei classificatori vengono combinati per determinare la classe finale di un caso di test.

13 Machine learning

13.1 Percettrone

Il percettrone è l'unità di base di una rete neurale. È composto da un vettore di input chiamato vettore delle feature, un vettore dei pesi che deve essere appreso, un termine di bias opzionale e un'operazione di somma pesata, un prodotto scalare tra il vettore delle feature e il vettore dei pesi. L'output della somma pesata passa attraverso una funzione di attivazione, tipicamente una funzione step nel caso del percettrone classico, che determina il risultato finale.

$$\text{activation}_w(x) = \sum_i w_i \cdot f_i(x) = w \cdot f(x)$$



13.2 Classificazione lineare

In un classificatore lineare, l'obiettivo è trovare un iperpiano che separi le istanze appartenenti a due classi distinte. L'iperpiano è definito dall'equazione:

$$w \cdot x + b = 0$$

Dove:

- x rappresenta il vettore delle feature
- w è il vettore dei pesi associati alle feature
- b è il termine di bias che consente di spostare l'iperpiano nello spazio

Le istanze di input, rappresentate come punti nello spazio delle feature, vengono classificate in base alla loro posizione rispetto a tale iperpiano:

- Se $w \cdot x + b > 0$ l'istanza è assegnata alla classe $Y = +1$
- Se $w \cdot x + b < 0$ l'istanza è assegnata alla classe $Y = -1$

13.3 Aggiornamento dei pesi

I pesi vengono inizializzati arbitrariamente, di solito ponendo i valori a 0. Per ogni istanza si fa una classificazione dei dati presenti, ovvero si cerca di capire in quale parte del piano si trovano. Se i dati si trovano nella porzione corretta del piano non è necessario effettuare alcuna ulteriore azione, se invece i dati si trovano nella parte sbagliata del piano è necessario spostarli in quella corretta andando a modificare il vettore dei pesi.

$$y = \begin{cases} +1 & \text{se } \mathbf{w} \cdot \mathbf{f}(x) \geq 0 \\ -1 & \text{se } \mathbf{w} \cdot \mathbf{f}(x) < 0 \end{cases}$$

- Se la classificazione è corretta (cioè, $y = y^*$), nessuna modifica!
- Se la classificazione è sbagliata: aggiusta il vettore dei pesi aggiungendo o sottraendo il vettore delle caratteristiche. Sottrai se y^* è -1.

$$\mathbf{w} = \mathbf{w} + y^* \cdot \mathbf{f}$$

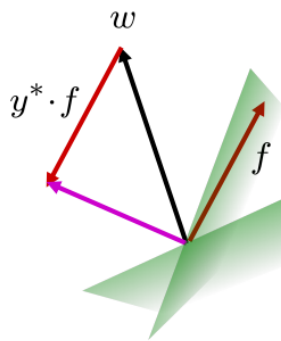


Figura 25: Vettore peso modificato

13.4 Regole di decisione multi-classe

Se ci sono più classi si utilizzano molteplici vettori peso, un vettore peso per ogni classe. L'attivazione si ottiene moltiplicando il vettore delle feature di ogni classe per il vettore dei pesi associato. Nel caso di classi multiple si avranno risultati diversi e andrà a selezionare la classe con il prodotto scalare più grande.

- Un vettore dei pesi per ogni classe:

$$w_y$$

- il vettore delle feature di una classe y :

$$w_y \cdot f(x)$$

- La classe con il valore del prodotto scalare più alto viene selezionata:

$$y = \arg \max_y w_y \cdot f(x)$$

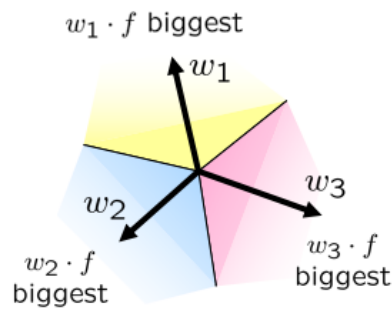


Figura 26: Esempio di classe multipla

13.5 Percettrone multi classe

Si inizia ponendo tutti i pesi a 0, si gestiscono i casi di addestramento uno a uno e si fa una previsione sui pesi. Se la previsione è corretta non si effettua nessun cambiamento, se la previsione è sbagliata è necessario modificare i pesi sia per l'output corretto che per l'output sbagliato. Vogliamo far sì che la risposta errata sia meno simile al vettore delle feature, mentre la risposta corretta vogliamo che sia più simile al vettore delle feature. Quindi si **sottrae** il valore del vettore delle feature dalla risposta sbagliata e si **addiziona** il valore del vettore delle feature alla risposta corretta. Per quanto riguarda gli altri vettori dei pesi non è necessario fare alcuna azione.

$$w_y = w_y - f(x)$$

$$w_{y^*} = w_{y^*} + f(x)$$

13.6 Proprietà del percettroni

Se i dati sono **separabili** e vengono separati allora il percettrone imparerà a conoscere la frontiera che li separa e la classificazione converge in una classificazione binaria. Quando i dati sono **non separabili** si può limitare il numero di errori che il modello può commettere in relazione alla separabilità dei dati. Tale limite è il seguente:

$$\text{Mistakes} = \frac{k}{\delta^2}$$

Dove:

- k : è una costante che dipende dal dataset, come la distanza tra i punti e la loro distribuzione.
- δ : è il margine di separazione.

13.7 Problemi del perceptrone

- **Rumore:** se i dati non sono separabili, il perceptrone non può trovare un iperpiano che separi perfettamente i dati. Pertanto potrebbe rimbalzare da una suddivisione all'altra. Una soluzione è considerare una divisione dell'iperpiano basata sulla media delle due divisioni precedenti.
- **Barely separating solution:** il perceptrone può trovare un iperpiano di separazione valido per il dataset ma non necessariamente quello ottimale.
- **Overtraining:** si verifica quando il perceptrone si adatta troppo bene ai dati di addestramento ma non generalizza in modo corretto i nuovi dati.

13.7.1 Dati non separabili: decisione deterministica

Quando i dati non sono linearmente separabili il perceptrone non è in grado di trovare un iperpiano che separi perfettamente le due classi. In questi casi potrebbe accadere che uno o più punti appartenenti a una classe si trovino dal lato sbagliato dell'iperpiano. Un classificatore deterministico, come il perceptrone, assegna una classe a ciascun punto in base alla posizione rispetto all'iperpiano senza essere in grado di gestire ambiguità.

13.7.2 Dati non separabili: decisione probabilistica

Quando i dati non sono separabili un classificatore probabilistico può essere più utile. Invece di fornire una decisione deterministica, un classificatore probabilistico fornisce una distribuzione di probabilità sulle possibili classi. Il classificatore predice la classe più probabile e fornisce una stima della probabilità che un punto appartenga a ciascuna delle classi possibili.

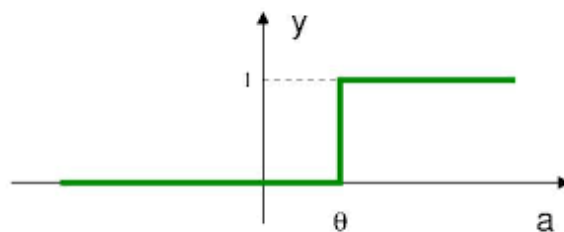
13.8 Decisioni deterministiche dei perceptroni

I perceptroni forniscono decisioni deterministiche, sia z il risultato del prodotto scalare tra il vettore dei pesi e il vettore delle feature:

- $z = w \cdot f(x)$
- Se $z = w \cdot f(x)$ è positivo $\rightarrow$ il classificatore dice: 1.0 di probabilità che la classe sia +1.
- Se $z = w \cdot f(x)$ è negativo $\rightarrow$ il classificatore dice: 1.0 di probabilità che la classe sia -1.

Step function:

$$H(z) = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$



- z = l'output del perceptrone
- $H(z)$ = è la probabilità che la classe sia +1 secondo il classificatore

13.8.1 Esempio sulle decisioni probabilistiche binarie

$$\frac{1}{1 + e^{-wx}}$$

Dove

- w è un vettore di pesi costanti che dobbiamo imparare
- wx è il prodotto scalare tra w e x
- Suppongo che $w = [-3, 4, 2]$ e $x = [1, 2, 0]$

Quale label sarà selezionata se si effettua una classificazione deterministica?

- $w \cdot x = -3 + 8 + 0 = 5$
- 5 è un valore positivo quindi il classificatore predice la label positiva

Qual è la probabilità di ciascuna label se si effettua una classificazione probabilistica?

- $1/(1 + e^{-5}) = 0.9933$ è la probabilità della label positiva
- $1 - 0.9933 = 0.0067$ è la probabilità della label negativa

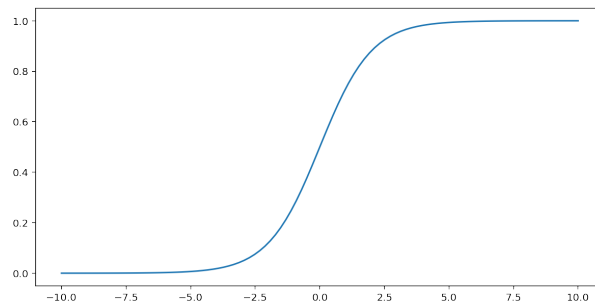
13.9 Decisioni probabilistiche dei perceptron

- $z = w \cdot f(x)$
- Se $z = w \cdot f(x)$ è molto positivo $\rightarrow$ la probabilità che la classe sia +1 dovrebbe essere vicina a 1.0.
- Se $z = w \cdot f(x)$ è molto negativo $\rightarrow$ la probabilità che la classe sia +1 dovrebbe essere vicina a 0.0.

Sigmoid function:

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

La funzione sigmoidea trasforma l'output z del percettrone in una probabilità che varia tra 0 e 1.



- z = è l'output del percettrone
- $\phi(z)$ = è la probabilità che la classe sia +1 secondo il classificatore

13.9.1 Esempio sulle decisioni probabilistiche multi-classe

- Suppongo che $w_1 = [-3, 4, 2]$, $w_2 = [2, 2, 7]$, $w_3 = [0, -1, 0]$ e $x = [1, 2, 0]$

Quale label sarà selezionata se si effettua una classificazione deterministica?

- $w_1 \cdot x = 5$, $w_2 \cdot x = 6$, $w_3 \cdot x = -2$
- $w_2 \cdot x$ ha il valore maggiore quindi il classificatore predice la seconda classe

Qual è la probabilità di ciascuna label se si effettua una classificazione probabilistica?

- Probabilità della classe 1: $\frac{e^5}{(e^5 + e^6 + e^{-2})} = 0.2689$
- Probabilità della classe 1: $\frac{e^5}{(e^5 + e^6 + e^{-2})} = 0.7310$
- Probabilità della classe 1: $\frac{e^5}{(e^5 + e^6 + e^{-2})} = 0.0002$

14 Deep learning

14.1 Regressione logistica

La regressione logistica è una tecnica utilizzata per problemi di classificazione binaria in cui l'obiettivo è prevedere se un'osservazione appartiene o meno a una delle due classi ($y = 0$, $y = 1$). La regressione logistica assume che la probabilità condizionata $P(y|\mathbf{x}; \theta)$, dove y corrisponde alla *label* (ovvero il valore che rappresenta la classe corretta dell'osservazione) e $\mathbf{x}$ è il vettore delle *feature*, possa essere modellata usando una **funzione sigmoidea**.

$$P(y^{(i)} = +1|x^{(i)}; w) = \frac{1}{1 + e^{-w \cdot f(x^{(i)})}}$$

$$P(y^{(i)} = -1|x^{(i)}; w) = 1 - \frac{1}{1 + e^{-w \cdot f(x^{(i)})}}$$

Dove:

- w rappresenta il vettore dei pesi
- $w^T x$ è il prodotto scalare tra il vettore dei pesi w e il vettore delle feature x

14.1.1 Funzione di perdita

L'obiettivo della regressione logistica è stimare i parametri w che massimizzano la probabilità di predire correttamente le etichette y per un insieme di dati $(x^{(i)}, y^{(i)})$. Questo si ottiene massimizzando la **log-likelihood**:

$$l(w) = \sum_{i=1}^n \log P(y^{(i)} | x^{(i)}; w),$$

dove la probabilità condizionata $P(y^{(i)} | x^{(i)}; w)$ è definita come:

$$P(y^{(i)} | x^{(i)}; w) = \begin{cases} \sigma(w^T x^{(i)}) & \text{se } y^{(i)} = 1, \\ 1 - \sigma(w^T x^{(i)}) & \text{se } y^{(i)} = 0, \end{cases}$$

e $\sigma(w^T x^{(i)}) = \frac{1}{1 + e^{-w^T x^{(i)}}}$ è la funzione sigmoidea.

Funzione costo

Trovare i parametri w che massimizzano la log-likelihood $l(w)$ è equivalente minimizzare la **funzione costo** definita come la log-likelihood negativa:

$$J(w) = -l(w) = - \sum_{i=1}^n [y^{(i)} \log \sigma(w^T x^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(w^T x^{(i)}))].$$

Questo approccio garantisce che i parametri w rendano il modello il più coerente possibile con i dati osservati.

14.2 Regressione logistica multi-classe

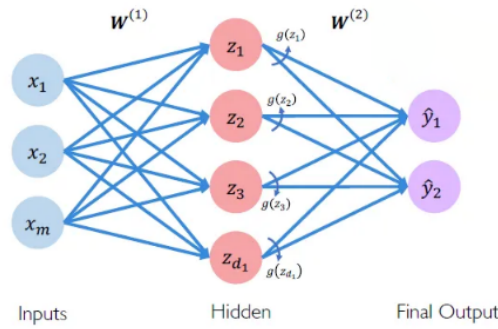
Nella regressione logistica multi-classe si vogliono classificare i punti di dati in K categorie distinte. Pertanto si punta a costruire un modello che fornisca delle stime di probabilità per un nuovo punto di appartenere a ciascuna delle K possibili categorie. Viene utilizzata la funzione **softmax** per modellare le probabilità di appartenenza a ciascuna delle K classi. La formula della funzione softmax è:

$$P(y^{(i)} | x^{(i)}, w) = \frac{e^{w_{y^{(i)}} \cdot f(x^{(i)})}}{\sum_y e^{w_y \cdot f(x^{(i)})}}$$

La funzione softmax prende un vettore di valori e li converte in una distribuzione di probabilità, dove ogni valore del vettore di output è una probabilità che somma a 1.

14.3 Rete neurale ad un livello

Una rete neurale a un livello è composta dalle seguenti sezioni principali:



1. Input Layer (Livello di Input)

- Riceve i dati di ingresso (feature) $x_1, x_2, \dots, x_m$.
- Ogni input rappresenta una caratteristica del dataset e viene propagato ai neuroni del livello nascosto tramite pesi $w_{j,i}^{(1)}$, dove i identifica un neurone nascosto e j un input.

2. Hidden Layer

- Contiene d_1 neuroni nascosti (ad esempio, $z_1, z_2, \dots, z_{d_1}$).
- Ogni neurone calcola una combinazione lineare degli input $\mathbf{x} = [x_1, x_2, \dots, x_m]$, applicando successivamente una funzione di attivazione g :

$$z_i = g \left(w_0^{(1)} + \sum_{j=1}^m x_j \cdot w_{j,i}^{(1)} \right)$$

dove:

- $w_{j,i}^{(1)}$ rappresenta i pesi tra l'input x_j e il neurone nascosto z_i ,
- $w_0^{(1)}$ è il bias per il neurone z_i , che consente flessibilità nella rappresentazione del modello,
- g è la funzione di attivazione che introduce non linearità.

3. Output Layer

- Contiene d_2 neuroni (ad esempio, $y_1, y_2, \dots, y_{d_2}$).
- Ogni neurone del livello di output riceve gli output dei neuroni nascosti $z_1, z_2, \dots, z_{d_1}$ e calcola una combinazione lineare di questi valori:

$$y_k = g \left(w_0^{(2)} + \sum_{i=1}^{d_1} z_i \cdot w_{i,k}^{(2)} \right)$$

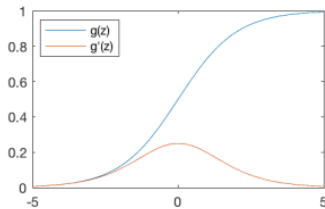
dove:

- $w_{i,k}^{(2)}$ sono i pesi tra il neurone nascosto z_i e il neurone di output y_k ,
- $w_0^{(2)}$ è il bias del neurone di output y_k ,
- Anche qui, g è una funzione di attivazione come sigmoid o softmax.

4. Output Finale

- La rete genera un output $\mathbf{y} = [y_1, y_2, \dots, y_{d_2}]$, che può rappresentare:
 - **Classi**: in problemi di classificazione,
 - **Valori continui**: in problemi di regressione.

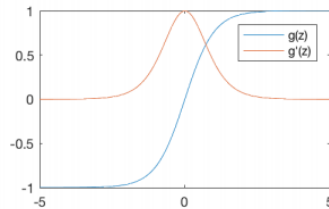
14.4 Funzioni di attivazione comuni



Sigmoid Function

$$g(z) = \frac{1}{1 + e^{-z}}$$

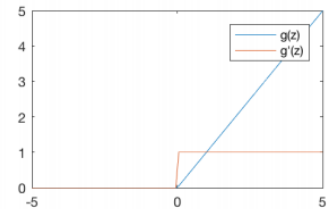
$$g'(z) = g(z)(1 - g(z))$$



Hyperbolic Tangent

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$



Rectified Linear Unit (ReLU)

$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

15 Addestramento delle reti neurali

15.1 Loss optimization

Nel processo di addestramento di una rete neurale, l'input x viene propagato attraverso i vari livelli della rete per calcolare l'output y . L'output finale viene confrontato con il valore reale $y^{(i)}$ utilizzando una funzione costo, che misura l'errore della rete. L'obiettivo dell'addestramento è trovare i valori dei pesi che minimizzano la funzione di costo:

$$W^* = \operatorname{argmin}_W \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; W), y^{(i)})$$

$$W^* = \operatorname{argmin}_W J(W)$$

Dove $J(W)$ rappresenta la funzione di costo aggregata su tutto il dataset.

Per minimizzare la funzione di costo, si utilizza un metodo iterativo basato sul calcolo del gradiente. Partendo da un punto iniziale W_0 si calcola il gradiente della funzione di costo rispetto ai pesi. Il gradiente indica la direzione di massima crescita della funzione;

pertanto, per ridurre l'errore, si aggiornano i pesi spostandosi nella direzione opposta al gradiente. La dimensione di ciascun passo è controllata dal parametro η noto come learning rate.

L'algoritmo più comune per questa operazione è la discesa del gradiente (*gradient descent*), che viene eseguita secondo i seguenti passi:

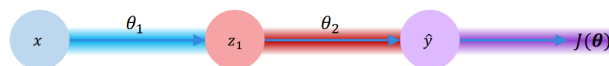
1. **Inizializzazione:** i pesi vengono inizializzati casualmente, ad esempio seguendo una distribuzione normale $W \sim \mathcal{N}(0, \sigma^2)$.
2. **Aggiornamento iterativo:** fino alla convergenza, si ripetono i seguenti passi:
 - (a) Calcolo del gradiente della funzione di costo rispetto ai pesi $\nabla J(W)$.
 - (b) Aggiornamento dei pesi nella direzione opposta al gradiente, con un passo determinato dal tasso di apprendimento η :

$$W \leftarrow W - \eta \nabla J(W)$$

3. **Termine dell'algoritmo:** l'aggiornamento viene ripetuto fino a raggiungere un punto di minimo locale o fino al soddisfacimento di un criterio di arresto, come una variazione minima della funzione di costo in un'iterazione o il raggiungimento di un numero massimo di iterazioni.

15.2 backpropagation

Per calcolare i gradienti per ogni livello della rete neurale si utilizza l'algoritmo di **backpropagation**. L'obiettivo è capire quanto la funzione costo sia sensibile al cambiamento dei pesi in modo tale da sapere quali modifiche possono garantire una più efficiente diminuzione del valore della funzione costo. Si utilizza la **chain rule** per propagare l'errore dall'output, attraverso gli hidden layer fino all'input.



$$\frac{\partial J(\theta)}{\partial \theta_1} = \frac{\partial J(\theta)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial z_1} * \frac{\partial z_1}{\partial \theta_1}$$

└───┘
└───┘
└───┘

Una volta calcolati i gradienti, i pesi della rete vengono aggiornati utilizzando il metodo del gradient descent.

15.3 Varianti di gradient descent

A seconda di come vengono utilizzati i dati, ci sono diverse varianti del gradient descent:

- **Stochastic Gradient Descent (SGD):** l'approccio stochastic Gradient Descent calcola il gradiente utilizzando un singolo campione di dati per ogni iterazione invece di utilizzare tutto il dataset. Richiede meno capacità computazionale rispetto al gradient descent standard e consente aggiornamenti frequenti dei parametri. L'utilizzo di un singolo dato introduce un rumore significativo.

- **Mini-Batch gradient descent:** l'approccio Mini-Batch gradient descent è un compromesso tra il gradient descent standard e lo stochastic gradient descent. Calcola il gradiente su piccoli gruppi di dati invece che su un unico punto o sull'intero dataset. Offre un'approssimazione accurata del vero valore del gradiente senza il costo computazionale dell'intero dataset, permette di iterare in modo veloce sul dataset e riduce la rumorosità rispetto a SGD. Inoltre consente la parallelizzazione della computazione grazie all'elaborazione di batch.

15.4 Problematiche di SGD

Quando la funzione di perdita cambia molto velocemente in una direzione (steep direction) e molto lentamente in un'altra (shallow direction), SGD può mostrare un comportamento oscillatorio. Durante l'ottimizzazione, i passi effettuati dall'algoritmo oscillano lungo la direzione ripida, mentre avanzano molto lentamente lungo la direzione più piatta. Una soluzione parziale potrebbe essere impostare un learning rate molto basso per mitigare l'oscillazione; tuttavia, ciò porterebbe ad una convergenza estremamente lenta. Inoltre, Se la funzione presenta minimi locali o punti di sella (punti in cui il gradiente si azzerava) SGD potrebbe rimanere bloccato. Sebbene SGD sia in grado di superare alcuni punti di minimo locale o di sella grazie alla sua natura stocastica rimane comunque sensibile in condizioni difficili.

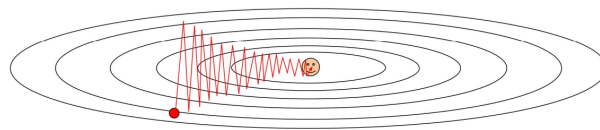


Figura 27: Esempio di problema SGD

15.5 SGD + momentum

SGD + momentum calcola un vettore di velocità che integra i valori del gradiente nel tempo. Il vettore velocità rappresenta una media mobile delle direzioni passate del gradiente. Ad ogni iterazione, il vettore di velocità v_t viene aggiornato combinando linearmente il vettore di velocità precedente, moltiplicato per un parametro di decadimento ρ e il gradiente corrente.

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

Dove ρ , solitamente, assume un valore tra $[0.9 - 0.99]$

$$x_{t+1} = x_t + \alpha v_{t+1}$$

Invece di seguire direttamente la direzione del gradiente calcolato nel punto corrente, si segue la direzione del vettore velocità v_t .

Minimi locali e punti di sella

Si può immaginare SGD + momentum come una pallina che rotola su una superficie, anche se il gradiente si annulla in un minimo locale, la “velocità accumulata” permette alla pallina di superare il punto di minimo e continuare il suo percorso. Lo stesso principio

si applica ai punti di sella, che sono comuni in spazi di alta dimensionalità. Un punto di sella è un punto in cui in una direzione la funzione obiettivo cresce mentre in un'altra direzione decresce. Grazie alla velocità accumulata, SGD con momentum riesce a uscire più facilmente da questi punti

Poor conditioning

In scenari in cui la funzione di perdita varia rapidamente in una direzione (ad esempio l'asse Y) e lentamente in un'altra (asse X). Il vettore velocità, che è una media mobile esponenzialmente ponderata dei gradienti precedenti, permette di ridurre le oscillazioni lungo l'asse Y dove la funzione varia rapidamente e aumenta l'efficienza lungo l'asse X dove la funzione varia lentamente.

Natura stocastica di SGD

La natura stocastica di SGD introduce rumore nei gradienti calcolati in ogni punto. SGD + momentum riduce questo rumore perché il vettore velocità si basa su una somma pesata di gradienti passati.

Aggiornamento del momentum

Il vettore verde rappresenta la velocità, ovvero la media dei gradienti osservata durante l'addestramento. Il vettore rosso rappresenta il gradiente calcolato nel punto corrente durante l'addestramento. La nuova velocità è ottenuta computando la media dei due vettori, il valore risultante è utilizzato per l'aggiornamento della matrice dei pesi.

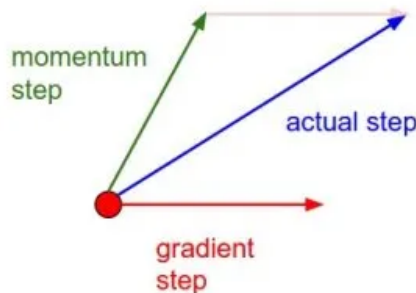


Figura 28: Momentum update

15.6 AdaGrad

AdaGrad è un algoritmo di ottimizzazione che appartiene alla famiglia degli algoritmi con **adaptive learning rates**. A differenza degli algoritmi classici che utilizzano un tasso di apprendimento fisso, AdaGrad tiene traccia della somma cumulativa dei quadrati dei gradienti per ogni parametro durante l'addestramento. Questo significa che, invece di aggiornare i parametri con un semplice gradiente, il passo di aggiornamento viene normalizzato dividendo per la radice quadrata della somma cumulativa dei quadrati dei gradienti.

$$G_t = G_{t-1} + (\nabla J(\theta_t))^2$$

Il principale limite dell'algoritmo AdaGrad consiste nel fatto che la somma cumulativa dei gradienti al quadrato G_t cresce in modo monotono durante l'addestramento. Poiché i gradienti al quadrato sono sempre positivi, G_t diventa sempre più grande, e di conseguenza il termine $\frac{\eta}{\sqrt{G_t + \epsilon}}$ diventa sempre più piccolo. Questo porta a una riduzione progressiva del learning rate che può diventare così piccolo da rendere gli aggiornamenti dei parametri praticamente nulli. L'algoritmo potrebbe sperimentare un arresto prematuro dell'ottimizzazione prima che si raggiunga un minimo della funzione di perdita.

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \cdot \nabla J(\theta_t)$$

RMSProp - Root Mean Square Propagation

RMSProp è stato sviluppato per risolvere il problema del learning rate decrescente di AdaGrad. In AdaGrad, la somma cumulativa dei gradienti al quadrato G_t cresce in modo monotono, causando una riduzione eccessiva del learning rate nel tempo. Ciò può portare a una convergenza molto lenta o addirittura a un arresto prematuro dell'ottimizzazione. RMSProp introduce una media esponenziale mobile dei gradienti al quadrato, dando più peso ai gradienti recenti e meno a quelli passati. Questo Approccio evita che il denominatore diventi troppo grande e assicura un tasso di apprendimento più stabile durante l'addestramento.

ADAM - Adaptive Moment Estimation

Adam è un algoritmo di ottimizzazione che combina due concetti chiave: il **momentum**, che stabilizza l'aggiornamento dei parametri utilizzando una media esponenziale dei gradienti passati e l'**adattività del learning rate**, che regola il passo di aggiornamento in base alla magnitudine dei gradienti, simile a RMSProp. Durante l'ottimizzazione, Adam calcola due quantità: il **primo momento**, che rappresenta una media dei gradienti, e il **secondo momento** che è una media dei gradienti al quadrato. Tuttavia, all'inizio del processo, questi momenti potrebbero essere distorti a causa dell'inizializzazione a zero portando potenzialmente a passi di aggiornamento troppo grandi. Per correggere questo problema, Adam introduce il **bias correction**, che aggiusta le stime dei momenti nei primi passi, garantendo che siano più accurate e affidabili.

15.7 Learning rate

Il learning rate è uno degli **iperparametri** più importanti nel deep learning poiché influisce direttamente sulla velocità e sulla stabilità del processo di addestramento. Se il learning rate è troppo alto il valore della loss può divergere tendendo a infinito. Al contrario, un learning rate troppo basso rallenta il training richiedendo molto tempo per raggiungere una loss ottimale. Quando non è possibile trovare un learning rate ottimale una strategia efficace è quella di iniziare con un valore relativamente alto per accelerare i progressi iniziali per poi ridurlo gradualmente durante il training. Questo approccio noto come **learning rate schedule**, combina i vantaggi di un apprendimento rapido iniziale con un affinamento più preciso nelle fasi successive.

- **Step Schedule:** lo step scehdule learning rate prevede di iniziare l'addestramento con un learning rate alto e di ridurlo dopo un certo numero di epoche o iterazioni. Questo approccio introduce diversi iperparametri aggiuntivi come il numero di iterazioni dopo le quali ridurre il learning rate e il nuovo valore di learning rate da applicare, rendendolo meno adatto a problemi nuovi o complessi dove non è possibile fare molti esperimenti.
- **Cosine learning rate schedule:** il cosine learning rate schedule utilizza una funzione coseno per determinare il valore del learning rate ad ogni epoca. Successivamente si sceglie la curva da seguire per la diminuzione del learning rate. Questo metodo richiede la scelta di solamente due iperparametri: α_0 , il learning rate iniziale e il numero di epoche di addestramento del modello.

$$\alpha_t = \frac{1}{2}\alpha_0(1 + \cos(t\pi/T))$$

- **Linear decay learning rate schedule:** il linear deacy learning rate schedule è un approccio che segue una curva lineare per il ridurre il valore del learning rate.

$$\alpha_t = \alpha_0(1 - t/T)$$

- **Inverse sqrt learning rate schedule:** è un metodo che riduce il learning rate seguendo una curva inversa della radice quadrata, permettendo un decadimento rapido all'inizio e una riduzione più lenta nelle fasi successive.

$$\alpha_t = \alpha_0/\sqrt{t}$$

Dove:

- α_0 : è il learning rate iniziale
- α_t : è il learning rate all'epoca t
- T : è il numero totale di epoche

15.8 Regularizzazione

La regularizzazione è una tecnica che consiste nell'aggiungere un termine di penalità alla funzione di perdita. Questo termine dipende dai pesi del modello e viene controllato da un iperparametro λ che determina l'importanza della regularizzazione rispetto alla perdita originale. Lo scopo principale è controllare la complessità del modello, impedendogli di adattarsi troppo ai dati di training e migliorando la sua capacità di generalizzare su dati nuovi e non visti. La funzione di perdita regularizzata ha la forma:

$$\text{Loss regularizzata} = \text{Loss originale} + \lambda \cdot \text{Termine di regularizzazione.}$$

Tipi di regolarizzazione

Esistono diversi tipi di regolarizzazione, tra cui:

- **Regolarizzazione L2:** la regolarizzazione L2 penalizza la somma dei quadrati dei pesi, riduce l'ampiezza dei pesi favorendo modelli con pesi piccoli e distribuiti in modo uniforme, ma non li azzerava. È utile in situazioni in cui si vuole evitare che i pesi diventino troppo grandi.
- **Regolarizzazione L1:** la regolarizzazione L1 penalizza la somma dei valori assoluti dei pesi, a differenza di L2, può portare alcuni pesi a zero, creando modelli sparsi, ovvero modelli in cui molti parametri sono esattamente zero, selezionando automaticamente le feature più rilevanti.
- **Elastic Net:** elastic net combina gli approcci L1 e L2, introducendo un parametro α per controllare il bilanciamento tra la regolarizzazione L1 e L2 nella funzione di perdita. Questo approccio è utile in situazioni in cui ci sono molte feature correlate.
- **Dropout:** Il dropout è una tecnica che consiste nel disattivare in modo casuale alcune attivazioni (output) dei neuroni durante l'addestramento. Ad esempio, con un valore di dropout pari al 50%, metà dei neuroni di uno strato saranno "spenti", (ossia le loro attivazioni saranno impostate a zero) in ogni forward pass. Questo processo cambia dinamicamente la capacità della rete neurale, poichè in ogni iterazione un diverso sottoinsieme di neuroni viene disattivato casualmente costringendo la rete a sviluppare percorsi ridondanti e robusti tra input e output migliorando la sua capacità di generalizzare.
- **Early stopping:** L'early stopping è una tecnica che si basa sull'analisi delle performance del modello durante l'addestramento, al fine di fermarlo prima che inizi l'overfitting. Durante l'addestramento si monitora la perdita (loss) sia sul dataset di addestramento che su quello di validazione.
 - **Training Loss:** Continua a diminuire man mano che il modello apprende.
 - **Validation Loss:** Dopo un determinato punto, smette di diminuire e inizia a crescere segnalando che il modello inizia a sperimentare overfitting.

Considerando un grafico, con le iterazioni di addestramento sull'asse X e la perdita sull'asse Y , il punto in cui il valore della perdita sul dataset di validazione inizia a salire è il momento ottimale per fermare l'addestramento.

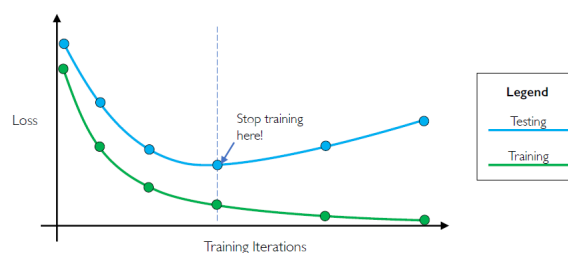


Figura 29: Early stopping

15.9 Overfitting

L'**overfitting** si verifica quando un modello di machine learning impara troppo bene i dati di training, memorizzando anche rumore e dettagli irrilevanti, ma fallisce nel generalizzare su nuovi dati. L'obiettivo è costruire modelli che siano in grado di apprendere dai dati di addestramento mantenendo una buona capacità di fare previsioni accurate su dati mai visti. Un modello che soffre di overfitting performa molto bene sui dati di training, ma ha prestazioni scarse sui dati di test. Al contrario, l'**underfitting** si verifica quando il modello non riesce ad adattarsi adeguatamente ai dati di training, ottenendo prestazioni mediocri sia sui dati di training che su quelli di test. L'obiettivo ideale è trovare un equilibrio, ovvero un modello che catturi le caratteristiche importanti dei dati senza memorizzare eccessivamente le sfumature, in modo da mantenere buone prestazioni sia sui dati di training che su quelli di test.



16 Scelta degli iperparametri

Questi sono i passaggi necessari per la scelta degli iperparametri quando non si ha a disposizione un grosso cluster di GPU.

1. **Step 1:** Il primo passaggio consiste nel controllare il valore iniziale della loss. Si disattiva il weight decay e si effettua un sanity check della loss all'inizializzazione. Ad esempio, per una funzione di perdita softmax con C classi, ci si aspetta un valore iniziale di $\log(C)$. Questo procedimento richiede una sola iterazione.
2. **Step 2:** Nel secondo step si cerca l'overfitting su un piccolo campione di dati di training: si considerano circa 5-10 mini-batches di dati e si cerca di raggiungere il 100% di accuratezza di training. Per fare ciò, si disattiva la regolarizzazione, si modifica l'architettura del modello (ad esempio, il numero di livelli e la dimensione di ogni livello), si varia il learning rate e l'inizializzazione dei pesi. Se la loss non diminuisce, il learning rate potrebbe essere troppo basso o l'inizializzazione potrebbe essere scarsa, al contrario se il valore della loss esplode a Inf il learning rate potrebbe essere troppo alto. Questa procedura è utile per individuare eventuali bug nel loop di ottimizzazione.
3. **Step 3:** Al terzo step si prende l'architettura dello step precedente, si utilizzano tutti i dati di addestramento e si cerca un learning rate che permetta alla loss di diminuire molto velocemente su tutti i dati di training. Si attiva un piccolo weight decay e si trova un learning rate che faccia diminuire la loss in modo significativo entro circa 100 iterazioni.
4. **Step 4:** Si crea una coarse grid per gli iperparametri, ad esempio si sceglie qualche valore per il learning rate e per il weight decay intorno a quelli che hanno funzionato

nello Step 3. Si addestrano pochi modelli per circa 1-5 epoche in modo tale da avere generalizzazione delle performance del modello.

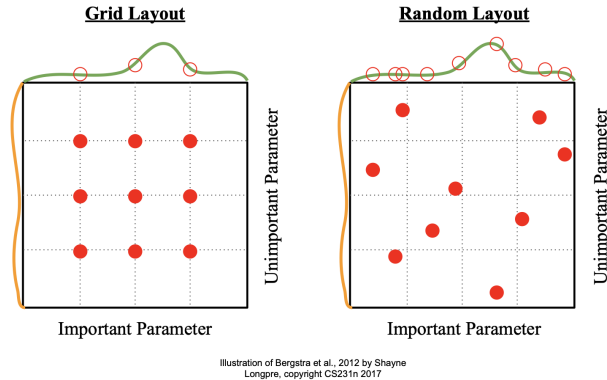
5. **Step 5:** Si esamina la griglia degli iperparametri, si selezionano i migliori modelli dello Step 4 e si addestrano nuovamente per un periodo più lungo, ad esempio per 10-20 epoche, senza learning rate decay.
6. **Step 6:** Si osservano le curve di loss e di accuratezza e, basandosi su queste, si fanno delle valutazioni su quali iperparametri modificare in futuro.

16.1 Grid search Vs random search

La scelta degli iperparametri è un aspetto cruciale nell'addestramento dei modelli di machine learning.

- **Grid search:** Una tecnica comune è grid search, in cui si seleziona un insieme di iperparametri da ottimizzare e per ciascuno di essi, si scelgono valori specifici da valutare. Per iperparametri come il learning rate o la regularization strength, è preferibile esplorare uno spazio log-lineare piuttosto che lineare. Se si hanno abbastanza risorse computazionali è possibile testare tutte le combinazioni di valori e selezionare quella che fornisce le migliori prestazioni. Tuttavia, questo approccio richiede risorse esponenziali rispetto al numero di iperparametri, rendendo grid search rapidamente impraticabile per problemi complessi.
- **Random search:** Un'alternativa più efficiente è random search, in cui si definiscono degli intervalli di valori per ciascun iperparametro e si campionano casualmente i valori all'interno di questi intervalli durante ogni addestramento. Per iperparametri come il learning rate o il weight decay, è preferibile utilizzare una distribuzione log-lineare, mentre per altri (ad esempio, la dimensione del modello o la probabilità di dropout) può essere più appropriata una distribuzione lineare. Random search è particolarmente vantaggioso quando si hanno molti iperparametri e non si sa in anticipo quali siano i più importanti per le prestazioni del modello.

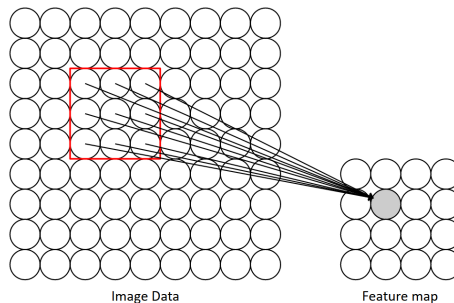
A differenza di grid search, che valuta sempre le stesse combinazioni di valori, random search esplora un'ampia gamma di combinazioni, fornendo una copertura più uniforme dello spazio degli iperparametri e permettendo di ottenere più informazioni utili da ogni addestramento. Inoltre, se un iperparametro è importante e un altro no, random search permette di campionare più efficacemente i valori dell'iperparametro importante, poiché i valori degli iperparametri meno importanti non sono allineati in modo rigido. Questo rende random search una scelta spesso preferibile, specialmente in contesti con molte dimensioni di iperparametri e risorse computazionali limitate.



17 Reti neurali convoluzionali CNN

17.1 Struttura spaziale

L'idea di base delle reti neurali convoluzionali è quella di connettere una **patch** nell'input (ad esempio una porzione 4×4 di pixel) a un singolo neurone nel layer successivo. Per fare ciò, si fa scivolare questa patch lungo l'immagine di input, producendo una nuova rappresentazione nell'hidden layer **chiamata feature map**. Questo approccio permette di mantenere la struttura spaziale dell'input e consente di individuare pattern visivi. Applicando dei pesi alle connessioni tra l'input e l'hidden layer, i neuroni del layer nascosto imparano a riconoscere feature specifiche in ciascuna patch.



17.2 Computazione

Ogni singolo neurone nell'hidden layer prende in input i valori dei neuroni corrispondenti alla patch nel livello precedente. Il filtro convoluzionale, rappresentato da una matrice di pesi, viene applicato alla patch tramite una moltiplicazione elemento per elemento, i prodotti calcolati al passo precedente vengono sommati. Questa somma restituisce un unico valore che rappresenta l'output del filtro per quella particolare patch. A questa somma viene aggiunto un bias e il valore risultante viene passato attraverso una funzione di attivazione non lineare.

$$\text{Output} = \sum_{i=1}^4 \sum_{j=1}^4 w_{ij} \cdot x_{(p+i, q+j)} + b$$

17.3 Convolutional layer

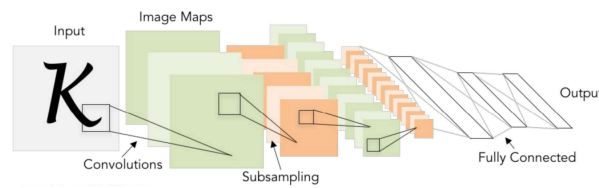


Figura 30: Architettura di una rete neurale convoluzionale

In un livello convoluzionale, possono essere applicati più filtri. La presenza di più filtri in un singolo livello convoluzionale permette alla rete di apprendere un insieme diversificato di feature (caratteristiche). L'output di un livello convoluzionale non è una singola immagine bidimensionale, ma un volume tridimensionale. Le dimensioni di questo volume sono:

- **Larghezza e altezza:** determinate dalle dimensioni spaziali dell'input, dalla dimensione del filtro, dallo stride e dal padding.
- **Profondità:** corrisponde al numero di feature maps generate, cioè al numero di filtri applicati al livello convolutivo.

Stride

Lo stride è un parametro che indica di quanto si sposta il filtro attraverso l'immagine di input. Un valore dello stride maggiore riduce le dimensioni spaziali dell'output, ad esempio uno stride pari a 2 comporta lo spostamento del filtro di due pixel alla volta riducendo la dimensione dell'output.

Padding

Il padding è una tecnica utilizzata per preservare le dimensioni spaziali dell'input durante l'operazione di convoluzione. Consiste nell'aggiungere una cornice di zeri attorno all'immagine di input. Le dimensioni dell'output dopo aver applicato il padding sono date da:

$$\text{Dimensioni output} = \frac{N + 2P - F}{\text{stride}} + 1$$

Dove:

- **N:** dimensione dell'input (la larghezza o altezza dell'immagine originale)
- **P:** padding
- **F:** dimensione del filtro di convoluzione
- **Stride:** il passo con cui si muove il filtro sull'immagine

Receptive field

Il campo recettivo in una rete neurale convoluzionale è la regione dell'immagine di input che influenza direttamente l'attivazione di un neurone in uno specifico livello della rete. Ogni neurone in uno strato convoluzionale non osserva l'intera immagine, ma solo una porzione limitata, il campo recettivo. Man mano che si procede negli strati più profondi della rete, il campo recettivo si espande, poiché ogni strato combina informazioni provenienti da regioni più ampie dell'input. La dimensione del receptive field dipende da:

- **Kernel size (K):** Il receptive field dipende dalla dimensione del kernel ($K \times K$) utilizzato nella convoluzione.
- **Layer addition (L):** Quando si aggiungono più strati convoluzionali, il receptive field cresce. Ogni strato aggiuntivo aumenta il receptive field di una quantità pari a $K - 1$.

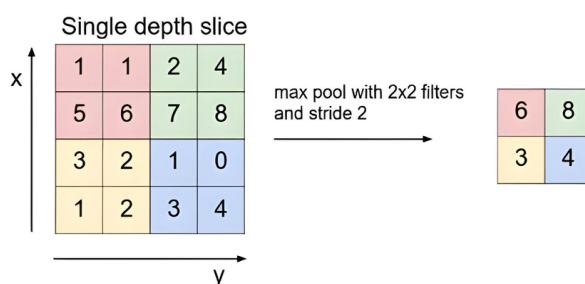
$$\text{Receptive field size} = 1 + L \times (K - 1)$$

Funzione di attivazione ReLU

Dopo l'operazione di convoluzione viene applicata una funzione di attivazione non lineare, tipicamente la ReLU. Questa funzione annulla i valori negativi nella feature map impostandoli a zero e lasciando invariati quelli positivi. Questa funzione introduce non linearità nel modello permettendo di apprendere pattern complessi.

Pooling

Il pooling è una tecnica utilizzata per ridurre le dimensioni spaziali delle feature map diminuendo il costo computazionale. La tecnica più comune è il **Max-Pooling** che seleziona il valore massimo all'interno di un filtro della feature map.



17.4 Transfer learning

Il transfer learning prevede, come primo passo, l'addestramento di una rete neurale convoluzionale su un dataset molto grande come ImageNet, che contiene milioni di immagini e migliaia di categorie. Questo addestramento iniziale permette alla rete di apprendere feature generiche che sono fondamentali per la comprensione delle immagini. Una volta che la rete è stata addestrata il passo successivo è capire che le performance su ImageNet non sono l'obiettivo finale, ma piuttosto un mezzo per ottenere una rete in grado di

estrarre feature utili per altri task o dataset più piccoli. Successivamente si può procedere in due modi:

- **feature extraction:** nel transfer learning, la feature extraction consiste nell'utilizzo di una rete pre-addestrata, come ad esempio una rete basata su ImageNet, eliminando l'ultimo fully connected layer. Questo layer, in reti come AlexNet, converte un vettore di feature a 4096 dimensioni in un vettore a 1000 dimensioni che rappresenta i punteggi di classe per le categorie su cui il modello è stato addestrato. Poiché l'ultimo layer è specifico per le classi originali, viene rimosso, e si utilizzano i vettori di feature a 4096 dimensioni prodotti dal penultimo layer come rappresentazione generale delle immagini. I pesi della rete vengono congelati, e queste feature estratte vengono impiegate come vettori rappresentativi per allenare modelli più semplici, come una regressione logistica o una support vector machine adattati al nuovo task. Questo approccio viene utilizzato se il nuovo dataset è abbastanza piccolo.
- **fine-tuning:** Se il dataset è molto grande si utilizza il fine tuning, si rimuove l'ultimo strato fully connected che viene rimpiazzato da un nuovo livello inizializzato per classificare le categorie del nuovo dataset. Il modello non viene utilizzato come semplice estrattore di feature, ma si utilizza la backpropagation per aggiornare i pesi non solo nel nuovo layer ma anche nei layer precedenti. Questa tecnica permette di adattare le rappresentazioni apprese dal modello al nuovo task migliorando le performance.

18 Apprendimento non supervisionato

L'apprendimento non supervisionato è un tipo di apprendimento automatico in cui l'algoritmo riceve in input dati non etichettati, ovvero senza un output desiderato.

18.1 Clustering

Il clustering è una tecnica di analisi dei dati che mira a raggruppare istanze di dati simili tra loro in gruppi, chiamati cluster, e a separare istanze dissimili in cluster diversi.

Caratteristiche del clustering

Gli algoritmi di clustering possono essere di diverse tipologie:

- **Algoritmi di clustering partizionale** come k-means, questi algoritmi dividono i dati in k cluster predefiniti.
- **Algoritmi di clustering basati sulla densità** come DBSCAN, questi algoritmi identificano cluster come regioni di alta densità di punti, separate da regioni di bassa densità.

Questi algoritmi utilizzano una funzione di distanza per determinare i raggruppamenti.

Qualità del clustering

- **Distanza inter-clustering:** per valutare in modo astratto la qualità di un raggruppamento la distanza inter-cluster dovrebbe essere massimizzata in modo che i cluster siano ben separati.
- **Distanza intra-clustering:** all'interno di ciascun cluster gli elementi dovrebbero essere il più vicini possibile, quindi la distanza intra-cluster dovrebbe essere minimizzata.

La qualità del clustering dipende da tre fattori:

1. Il tipo di algoritmo
2. La funzione di distanza utilizzata
3. L'applicazione specifica del problema

18.2 Clustering Vs classificazione

Entrambi gli approcci hanno a che fare con dei sottoinsiemi di un insieme più grande e si occupano di assegnare un elemento a uno di questi sottoinsiemi.

Classificazione

- **Classi predefinite:** le classi devono essere conosciute prima di applicare l'algoritmo di classificazione.
- **Esempi etichettati:** è necessario disporre di un numero significativo di dati etichettati per addestrare il classificatore.
- **Obiettivo:** lo scopo è assegnare nuovi elementi a una delle classi già definite.
- **Valutazione:** è possibile trattenere parte dei dati etichettati per valutare le prestazioni del classificatore.

Clustering

- **Classi sconosciute:** non è necessario sapere a priori quante classi sono presenti nel dataset. Questo dipende dall'algoritmo di clustering utilizzato.
- **Dati non etichettati:** si lavora con dati non etichettati, e l'obiettivo è scoprire raggruppamenti o strutture nei dati.
- **Interpretazione:** i cluster trovati potrebbero non avere una chiara interpretazione iniziale.
- **Valutazione complicata:** misurare la qualità del clustering è più complesso rispetto alla classificazione.

Differenze chiave

La classificazione e il clustering differiscono principalmente per il loro approccio al problema. La classificazione richiede una comprensione chiara del problema e dati etichettati, al contrario, il clustering è un metodo esplorativo che richiede solo competenze geometriche per descrivere e misurare oggetti nello spazio del problema. Inoltre, il clustering può essere utilizzato nelle fasi iniziali di analisi per suggerire possibili classi e fungere da base per un successivo approccio di classificazione.

18.3 K-means

K-means è un algoritmo di clustering partizionale. Sia D un insieme di n punti dati o istanze rappresentato come $D = \{x_1, x_2, \dots, x_n\}$. Ogni punto dato $x_i = \{x_{i1}, x_{i2}, \dots, x_{ir}\}$ è un vettore nello spazio $X \subseteq R^r$ dove r è il numero degli attributi. k rappresenta il numero di cluster in cui si vogliono dividere i dati ed è un valore specificato dall'utente. Ogni cluster ha un centro chiamato **centroide**.

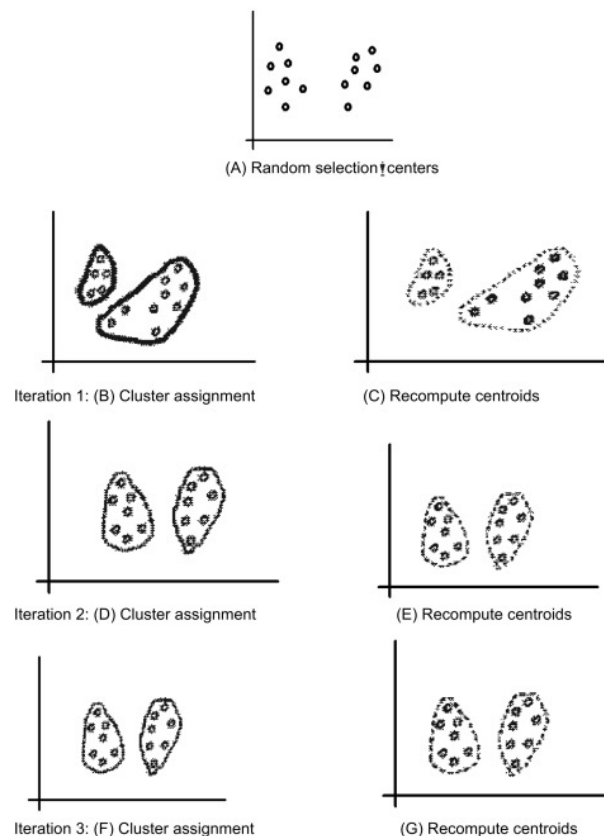


Figura 31: Esempio di funzionamento dell'algoritmo k-means

Funzionamento

1. Si selezionano casualmente k punti dati dal dataset. Questi punti rappresentano i centroidi iniziali dei cluster, ovvero i punti di partenza intorno ai quali si formeranno i gruppi.

2. Per ogni punto dati nel dataset, si calcola la distanza da ciascuno dei k centroidi, ogni punto viene assegnato al cluster il cui centroide è più vicino.
3. Una volta che tutti i punti sono stati assegnati a un cluster, si ricalcolano le posizioni dei centroidi.
4. Se non viene rispettato alcun criterio di convergenza l'algoritmo continua iterativamente dal punto 2.

Algorithm 2 K-means(k, D)

- 1: Choose k data points as the initial centroids (cluster centers)
 - 2: **repeat**
 - 3: **for** each data point $x \in D$ **do**
 - 4: Compute the distance from x to each centroid
 - 5: Assign x to the closest centroid {A centroid represents a cluster}
 - 6: **end for**
 - 7: re-compute the centroids using the current cluster memberships
 - 8: **until** the stopping criterion is met
-

Criteri di convergenza

- Se i punti dati non vengono più riassegnati a cluster diversi, l'algoritmo può fermarsi.
- Se il cambiamento nella posizione dei centroidi è minimo o nullo allora l'algoritmo converge.
- L'algoritmo converge quando si verifica una minima diminuzione della somma degli errori al quadrato. SSE è una metrica di valutazione della coesione dei cluster.

$$SSE = \sum_{j=1}^k \sum_{x \in C_j} \text{dist}(x, m_j)^2$$

Dove C_j è il j -esimo cluster, m_j è il centroide del cluster C_j e $\text{dist}(x, m_j)$ è la distanza tra il punto dato x e il centroide m_j .

Funzione di distanza

L'algoritmo *k-means* può essere utilizzato per qualsiasi dataset applicativo in cui la **media** può essere definita e calcolata. Nello *spazio euclideo*, la media di un cluster si calcola come:

$$m_j = \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

dove $|C_j|$ è il numero di punti dati nel cluster C_j . La distanza di un punto dati x_i da un centroide m_j si calcola con:

$$\begin{aligned} \text{dist}(x_i, m_j) &= \|x_i - m_j\| \\ &= \sqrt{(x_{i1} - m_{j1})^2 + (x_{i2} - m_{j2})^2 + \dots + (x_{ir} - m_{jr})^2} \end{aligned}$$

18.4 Punti di forza e di debolezza di k-means

Punti di forza

- L'algoritmo è facile da capire e da implementare.
- L'algoritmo è efficiente in termini di complessità computazionale: ha una complessità $O(tkn)$, dove n è il numero di punti dati, k è il numero di cluster e t è il numero di iterazioni. Poiché spesso k e t sono piccoli, k-means è considerato un algoritmo lineare rispetto al numero di dati n .
- L'algoritmo non garantisce di raggiungere l'ottimo globale, ma se si utilizza la somma degli errori quadratici converge a un ottimo locale.

Punti di debolezza

- L'algoritmo può essere applicato solo se la media è definita.
- L'utente deve specificare il numero di cluster k , che non è sempre noto a priori.
- L'algoritmo è sensibile alla scelta iniziale dei centroidi.
- L'algoritmo è sensibile agli outliers, ovvero i punti di dati che sono molto distanti dagli altri punti di dati.

18.5 Gestione degli outlier

- Una tecnica consiste nel rimuovere dei punti dal dataset, in particolare i punti che sono molto lontani dai centroidi rispetto agli altri punti del dataset. Per essere sicuri della scelta si monitorano questi possibili outliers su più iterazioni e successivamente si decide se rimuovere tali punti oppure no.
- Un altro approccio consiste nell'utilizzare un **sampling randomico**, con il sampling si va a considerare solo un piccolo sottoinsieme del dataset e quindi la probabilità di selezionare un outlier è molto bassa. Tutti i punti che non sono stati considerati nel clustering potrebbero essere assegnati per distanza dal centroide.

Inoltre l'algoritmo k-means funziona meglio con cluster convessi e di forma simile, potrebbe riscontrare difficoltà con distribuzioni più complesse come quelle annidate o con forme irregolari.

18.6 Valutazione dei cluster

La valutazione della qualità del clustering è un processo complesso perché non si conoscono i cluster corretti a priori. In assenza di dati etichettati, si possono adottare metodi qualitativi come l'ispezione manuale dei risultati, l'osservazione della posizione dei centroidi e della distribuzione dei cluster. Con dati etichettati, invece, il clustering può essere trattato come una classificazione, assumendo che ogni classe corrisponda a un cluster. In questo caso, è possibile costruire una matrice di confusione per calcolare metriche come entropia, purezza e precisione. La valutazione della qualità del clustering può essere effettuata

anche ricorrendo a tecniche di valutazione interne che misurano la coesione intra-cluster, quanto sono vicini i punti all'interno di un cluster, e la separazione tra cluster, i centroidi di cluster differenti dovrebbero essere lontani gli uni dagli altri.

18.7 Clustering basato su densità

I metodi di clustering basati sulla densità individuano regioni dense di punti nello spazio dei dati separate da aree a bassa densità. Diversamente da approcci come k-means, non richiedono la definizione a priori del numero di cluster. La loro capacità di rilevare cluster di forma arbitraria li rende particolarmente versatili. Tuttavia richiedono la definizione di diversi iperparametri e sono più sensibili ai valori degli stessi.

18.8 Definizione di densità

- **ϵ -Vicinato:** dato un valore ϵ scalare, l' ϵ -vicinato di un punto p è l'insieme dei punti q tali che distano ϵ da p .

$$N_\epsilon(p) = \{q \mid d(p, q) \leq \epsilon\}$$

- **Alta densità:** Un ϵ -vicinato è definito ad alta densità se contiene almeno $MinPts$ punti.

Core-Border-Outlier

Dati ϵ e $MinPts$ è possibile categorizzare gli oggetti in 3 gruppi diversi:

- **Core point:** un punto è un core point se ha più di un numero specifico di $MinPts$ entro ϵ , questi sono i punti all'interno di un cluster.
- **Border point:** un punto è un border point se ha meno di $MinPts$ punti nel suo ϵ -vicinato, ma si trova nel vicinato di un core point.
- **Noise point:** un qualunque altro punto che non è un core point o un border point è un noise point.

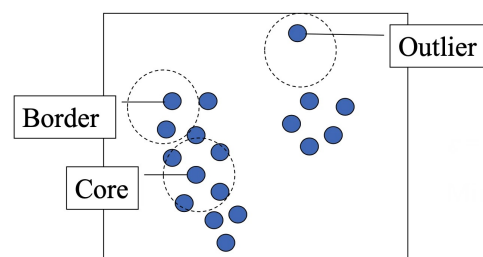


Figura 32: $\epsilon = 1$ unit, $MinPts = 5$

Densità-raggiungibilità

Si considera un punto di partenza p , un altro punto q è raggiungibile per densità da p in maniera **diretta** o **indiretta**.

- **Raggiungibilità per densità diretta:** un punto q è direttamente raggiungibile da p se p è un core point e q si trova nell' ε -vicinato di p . Questa relazione non è simmetrica, poiché p deve essere un core point, mentre q non necessariamente lo è.
- **Raggiungibilità per densità indiretta:** un punto q è raggiungibile indirettamente da p se esiste una catena di punti $(p_1, p_2, \dots, p_k)$ tale che $p_1 = p$ e $p_k = q$ e ogni punto nella catena è direttamente raggiungibile per densità dal precedente. Questo consente di collegare punti che non sono direttamente raggiungibili da p attraverso intermediari.

È importante scegliere con attenzione i valori degli iperparametri ε e $MinPts$, un ε troppo grande rischia di includere troppi punti in un unico cluster, mentre un valore troppo piccolo potrebbe impedire la formazione di cluster.

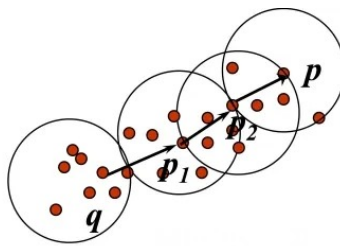


Figura 33: $MinPts = 7$

18.9 Potenzialità e difetti di DBSCAN

DBSCAN è un algoritmo resistente al rumore, in grado di identificare outlier e di gestire cluster di diversa forma, dimensione, cardinalità ed estensione spaziale se opportunamente calibrato. Tuttavia è molto sensibile ai parametri ε e $MinPts$, ha difficoltà nel determinare un corretto set di parametri e fatica a gestire situazioni in cui la densità dei dati varia molto.

Algorithm 3 DBSCAN Algorithm

Require: D (dataset), ϵ (radius), $minPts$ (minimum number of points)

Ensure: $clusters$ (list of clusters), $noise$ (set of noise points)

```
1:  $clusters \leftarrow []$ 
2:  $visited \leftarrow \emptyset$ 
3:  $noise \leftarrow \emptyset$ 
4: for each point  $d$  in  $D$  do
5:   if  $d \in visited$  then
6:     continue
7:   end if
8:    $visited.add(d)$ 
9:    $neighbors \leftarrow get\_neighbors(D, d, \epsilon)$ 
10:  if  $size(neighbors) \geq minPts$  then
11:     $new\_cluster \leftarrow []$ 
12:     $expand\_cluster(d, neighbors, new\_cluster, visited, \epsilon, minPts)$ 
13:     $clusters.append(new\_cluster)$ 
14:  end if
15: end for
16: for each point  $d$  in  $D$  do
17:   if  $d \notin$  any cluster in  $clusters$  then
18:      $noise.add(d)$ 
19:   end if
20: end for
21: return  $clusters, noise$ 
```
